CHAPTER 3

How Bitcoin Works

Blockchain technology is all the rage these days, thanks to Bitcoin! blockchain as we know it is a gift of Bitcoin and its inventor, Satoshi Nakamoto, to the whole world. If you are wondering who Satoshi Nakamoto is, it is the name used by the unknown person or persons who originated Bitcoin. We suggest that you understand and appreciate the wonderful technology behind Bitcoin without searching for the inventor. Learning the technical fundamentals of Bitcoin will enable you to understand the other blockchain applications that are there in the market.

Since Bitcoin testified to the robustness of blockchain technology for years, people now believe in it and have started exploring other possible ways to use it. In the previous chapter, we already got the hang of how blockchain works at a technical level, but learning Bitcoin can give you the real taste of blockchain. You may want to consider Bitcoin as a cryptocurrency use case of blockchain technology. So, this chapter will not only help you understand how Bitcoin works in particular, but also give you a perspective of how different use cases can be built using blockchain technology, the way Bitcoin is built.

We will cover Bitcoin in greater detail throughout this chapter and while doing so, a lot of blockchain fundamentals will also be clarified with more practical insights. If you are already familiar with the Bitcoin fundamentals, you may skip this chapter. Otherwise, we advise you to follow through the concepts in the order presented. This chapter explains what Bitcoin is, how it is designed technically, and provides an analysis of some inherent strengths and weaknesses of Bitcoin.

© Bikramaditya Singhal, Gautam Dhameja, Priyansu Sekhar Panda 2018
B. Singhal et al., *Beginning Blockchain*, https://doi.org/10.1007/978-1-4842-3444-0_3

The History of Money

Ever wonder what money is and why it even exists? Money is primarily the medium of exchange for exchanging value, that is anything of value. It has a history to it. We will quickly recap the history to be able to understand how money has eveolved to how we know it today, and how Bitcoin furthers it to the next level.

Not everyone has everything. In the good old days when there were no notions of currency or money, people still figured out how they could exchange what they had in surplus for what they needed from someone else. Those were the days of the barter system. Wheat in exchange for peddy or oranges for lemons was the system. This was all good, but what if someone having wheat needs medicine that the other person does not have? Example: Alice has wheat and needs medicine, but Bob knows she has access to someone who has oranges, and Bob needs wheat. In this situation, the exchange is not working out. So, they have to find a third person, Charlie, who might need oranges and as well has surplus medicine. A pictorial representation of this scenario is shown in Figure 3-1.

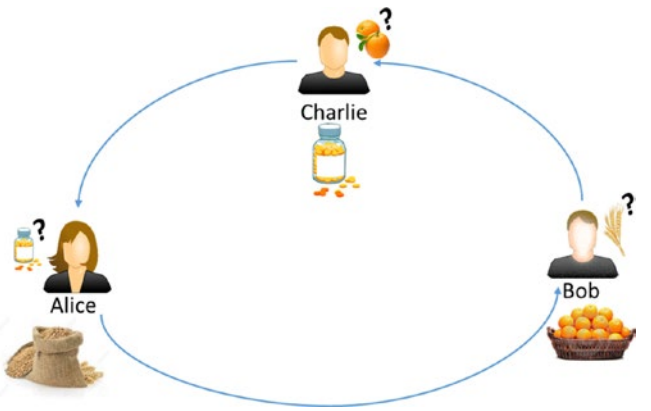


Figure 3-1. *The primitive barter system*

It was always tough to find a person such as Charlie in the previous example who could fit in the puzzle so easily; this problem had to be solved. So, people started thinking of a commoditized system of value exchange. There were a few items that everyone would need, such as milk, salt, seeds, sheep, etc. This system almost worked out! Soon after, people realized that it was quite inconvenient and difficult to store such commodities.

Eventually, better techniques were found to be used as financial instruments, such as metal pieces. People valued the rare metals more than the usual ones. Gold and silver metals topped the list as they wouldn't corrode. Then countries started minting their own currency (metal coins with different weights) with their official seal in them. Though the metal pieces and coins were better than the previous system, as one could easily store and carry them, they were vulnerable to theft. Temples came into rescue as people trusted in them and had a strong belief that no one would steal from temples. The priests would give a receipt to the person depositing gold that would mention the amount of gold/silver received, as a promise to acknowledge their deposit and give back to the bearer of the receipt the same when they returned. The person bearing the receipt could circulate the receipt in the market to get what they wanted. This was the beginning of our banking system. The receipt worked, as the fiat currency and the temples played the role of centralized banks that people trusted. Refer to Figure 3-2 to understand how this system appeared back then.

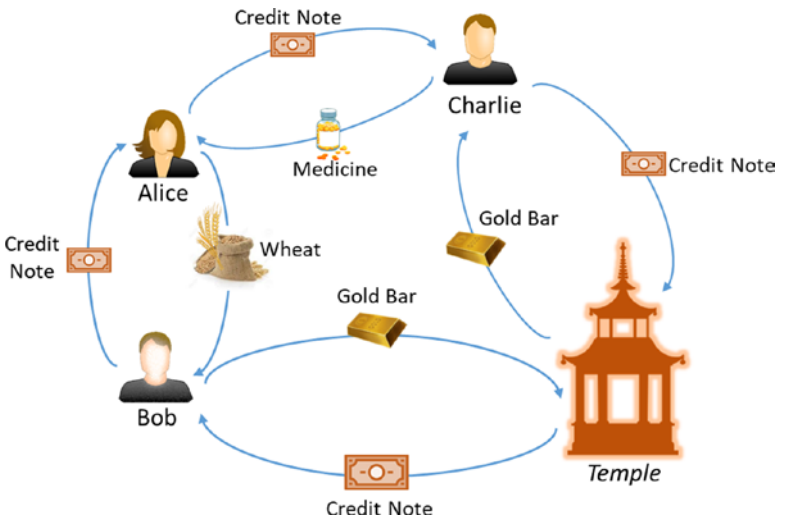


Figure 3-2. *The beginning of the banking era*

In the system just mentioned, currency was always backed by some precious metal such as gold or silver. This system continued even after the goverenments and banks replaced the temples. This is how the commodity currency came up in the market to enable a universal medium of value exchange for the goods and services. Whatever currency was there in those days was all backed by gold/silver.

Slowly, "fiat currency" was introduced by the governments as legal tender, which was no longer backed by gold or silver. It was purely trust based, in the sense that people did not have a choice but to trust the government. Fiat currency does not have any intrinsic value, it is just backed by the government. Today, the money that we know of is all fiat currencies. So, the value of money today depends on the stability and performance of the governments in whose jurisdiction the currency is being issued and used. Those paper currencies were the money themselves and there was nothing more valubale in the banks. This was the state of banking systems and at the same time the digital world was just forming up.

Around the 1990s, the Internet world was gaining momentum and the banking systems were getting digitized. Since some level of discomfort was still there with fiat currencies, since they were perishable and vulnerable to theft, banks assured that people could just go digital! This was the era when even the paper notes weren't required to be printed. Money became the digital numbers in the computer systems of banks. Today, if every account holder went to their respectve bank and demanded the currency notes for the amount of money they hold in their accounts, the banks would be in big trouble! The total real money in circulation is extremely marginal compared with the amount of digital money worldwide.

Dawn of Bitcoin

In the first chapter we looked at the technology aspects of the Internet revolution, and in the previous section of this chapter we looked at the evolution of money. We should now look at them side by side to understand Satoshi Nakamoto's perspective behind designing Bitcoin–a cryptocurrency. In this section and elsewhere in this text, we will try to elaborate on Satoshi's statements in the paper he wrote on Bitcoin.

We learned about temples and then governments and banks for the role they played in the currency systems that eveolved from barter systems. Even today, the situation is just the same. If you zoom in a bit on these systems, you will find that the one pivotal thing that makes these systems stable is the "trust" element. People trusted temples, and then they trusted governments and banks. The entire commerce on the Internet today relies on the centralized, trusted third parties to process payments. Though the Internet was designed to be peer-to-peer, people build centralized systems on it to reflect the same old practice. Well, technically building a peer-to-peer system back in the 2000s was quite tough considering the maturity of technology during that time. Consequently, the cost of transactions, time taken for a transaction to settle, and other issues due to centralization were obvious. This wasn't the case with physical currencies, as transactions meant settlement.

Could there be a digital currency backed by computing power, the same way gold was used to back the money in circulation? The answer is "Yes," thanks to Satoshi's Bitcoins. Bitcoins are designed to enable electronic payments between two parties based on cryptographic proof, and not based on trust due to intermediary third parties. It is possible today because of the technological advancements. In this chapter, we will see how Satoshi Nakamoto combined cryptography, game theory, and computer science engineering fundamentas to design the Bitcoin system in 2008. After it went live in 2009 and till today, the system is quite stable and robust enough to sustain any kind of cyber attacks. It stood the test of time and positioned itself as a global currency.

What Is Bitcoin?

Blockchain offers cryptocurrency: digital money! Just as we can transact with physical currency without banks or other centralized entities, Bitcoin is designed to facilitate peer-to-peer monetary transactions without trusted intermediaries. Let us look at what it is and then learn later in the chapter how it really works. Bitcoin is a decentralized cryptocurrency that is not limited to any nation and is a global currency. It is decentralized in every aspect–technical, logical, as well as political. As and when the transactions are validated, new Bitcoins get mined and a maximum of 21 million Bitcoins can ever be produced. Approximately, to reach 21 million Bitcoins, it would be take until the year 2140. Anyone with good computing power can participate in mining and generate new Bitcoins. After all the Bitcoins get generated, no new coins can be minted and only the ones in circulation would be used. Note that Bitcoins do not have fixed denominations such as the national fiat currencies. As per design, the Bitcoins can have any value with eight decimal places of precision. So, the smallest value in Bitcoin is 0.00000001 BTC, which is called 1 Satoshi.

The miners mine the transactions to mint new coins and also consume the transaction fee that the person willing to make a transaction is ready to pay. When the total number of coins reaches 21 million, the miners would validate the transactions solely for the transaction fees. If someone tries to make a transaction without a transaction fee, it may still get mined because it is a valid transaction (if at all it is) and also the miner is more interested in the mining reward that lets him generate new coins.

Are you wondering what decides the value of Bitcoins? When currency was backed by gold, it had great significance and was easy to assess the value based on gold standards. When we say Bitcoin is backed by the computing power that people use for mining, that is not enough to understand how it attains its value. Here is a litle bit of economics required to understand it.

When fiat currency was launched for the first time, it was backed by gold. Since people believed in gold, they believed in currency as well. After a few decades, currency was no longer backed by gold and was totally dependent on the governments. People continued believing in it because they themselves form or contribute to the formation of their own government. Since the governments ensure its value, and people trust it, so it attains that value. In an international setting, the value of currency of specific countries depends on various factors and the most important of them is "supply and demand." Please keep in mind that some countries that printed a lot of fiat currency notes went bankrupt; their economy went down! There has to be a balance and to understand this, more economics is needed, which is beyond the scope of this book. So, let us get back to Bitcoins for now.

When Bitcoin was first launched, it did not have any official price or value that people would believe. If one would sell it for some US dollars (USD), I would never have bought those initially. Gradually when the exchange started taking place, it developed a price and one Bitcoin was

not even one USD then. Since Bitcoins are generated by a competitive and decentralized process called "mining," and they are generated at a fixed rate with an upper cap of 21 million Bitcoins in total that can ever exist, this makes Bitcoin a scarce resource. Now relating this context back to the game of "supply and demand," the value of Bitcoin started inflating. Slowly, when the entire globe started believing in it, its price even skyrocketed from a few USDs to thousands of USDs. Bitcoin adoption among the users, merchants, start-ups, big businesses, and many others is growing like never before because they are being used in the form of money. So, the value of Bitcoin is highly influenced by "trust," "adoption," and "supply and demand" and its price is set by the market.

Now, the question is why the value of Bitcoin is so volatile as of this writing and fluctuates quite a lot. One obvious reason is supply and demand. We learned that there can only be a limited number of Bitcoins in circulation, which is 21 million, and the rate at which they get generated is decreeasing with time. Because of this design, there is always a gap in supply and demand, which results in this volatility. Another reason is that Bitcoins are never traded in one place. There are so many exchanges in so many places across the globe, and all those exchanges have their own exchange prices. The indexes that you see gather Bitcoin exchange prices from several exchanges and then average them out. Again, since all these indexes do not collect data from the same set of exchanges, even they do not match. Similarly, the liquidity factor that implies the amount of Bitcoins flowing through the entire market at any given time also influences the volatility in Bitcoin price. As of now, it is definitely a high-risk asset but may get stabilized with time. Let us take a look at the following list of factors that may influence the supply and demand of Bitcoins, and hence their price:

- Confidence of people in Bitcoin and fear of uncertainty
- Press coverage with good and bad news on Bitcoin

- Some people own Bitcoins and do not allow them to flow through the market and some people keep buying and selling to minimize risk. This is why the liquidity level of Bitcoin keeps on changing.
- Acceptance of Bitcoins by big ecommerce giants
- Banning of Bitcoins in specific countries

If you are now wondering if there is any possibility of Bitcoin to crash completely, then the answer is "Yes." There are many examples of countries whose currency systems have crashed. Well, there were political and economic reasons for them to crash such as hyperinflation, which is not the case with Bitcoins because one cannot generate as many Bitcoins as they want and the total number of Bitcoins is fixed. However, there is a possibility of technical or cryptographic failure of Bitcoins. Please note that Bitcoin has stood the test of time since its inception in 2008 and there is a possibility that it will grow much bigger with time, but it cannot be guaranteed!

Working with Bitcoins

In order to get started with Bitcoins, no technicality is needed. You just have to download a Bitcoin wallet and get started with it. When you download and install a wallet on your laptop or mobile, it generates your first Bitcoin address (public key). You can generate many more, however, and you should. It is a best practice to use the Bitcoin addresses only once. Address reuse is an unintended practice in Bitcoin, though it works. Address reuse can harm privacy and confidentiality. As an example, if you are reusing the same address, signing a transaction with the same private key, the recipient can easily and reliably determine that the address being reused is yours. If the same address is used with multiple transactions, they can all be tracked and finding who you are gets even easier.

Remember that Bitcoin is not fully anonymous; it is said to be pseudonymous and there are ways to trace the transaction origins that can reveal the owners.

You have to disclose your Bitcoin address to the person willing to transfer Bitcoins to you. This is very safe because the public key is public anyway. We know that there is no notion of a closing balance in Bitcoin and all records are there as transactions. Bitcoin wallets can easily calculate their spendable balance, as they have the private keys of the corresponding public keys on which transactions are received. There are a variety of Bitcoin wallets available from so many wallet providers. There are mobile wallets, desktop wallets, browser-based web wallets, hardware wallets, etc., with varying levels of security. You need to be extremely careful in the wallet security aspect while working with Bitcoins. The Bitcoin payments are irreversible.

You must be wondering how secured are these wallets. Well, different wallet types have different leves of security and it depends on how you want to use it. Many online wallet services suffered from security breaches. It is always a good practice to enable two-factor authentication whenever applicable. If you are a regular user of Bitcoins, it may be a good idea to use small amounts in your wallets and keep the remainder separately in a safe environment. An offline wallet or cold wallet that is not connected to the network provides the highest level of security for savings. Also, there should be proper backup mechanisms for your wallet in case you lose your computer/mobile. Remember that if you lose your private key, you lose all the money associated with it.

If you have not joined Bitcoin as a miner running a full node, then you can just be a user or a trader of Bitcoins. You will definitely need an exchange from where you can buy some Bitcoins with your US dollars or other currencies as accepted by the exchanges. You should prefer buying Bitcoins from a legitimate and secured exchange. There have been many examples of exchanges that suffered from security breaches.

The Bitcoin Blockchain

We already looked at the basic blockchain data structure in the previous chapter and also covered the basic building blocks of a blockchain data structure such as hashing techniques and asymmetric cryptography. We will learn the specifics of Bitcoin blockchain in this section.

The Bitcoin blockchain, like any other blockchain, has a similar blockchain data structure. The Bitcoin Core client uses the LevelDB database of Google to store the blockchain datastructure internally. Each block is identified by its hash (Bitcoin uses the SHA256 hashing algorithm). Every block contains the hash of the previous block in its header section. Remember that this hash is not just the hash of the previous header but the entire block including header, and it continues all the way to the *genesis* block. The genesis block is the beginning of any blockchain. Typically, a Bitcoin blockchain looks as shown in Figure 3-3.

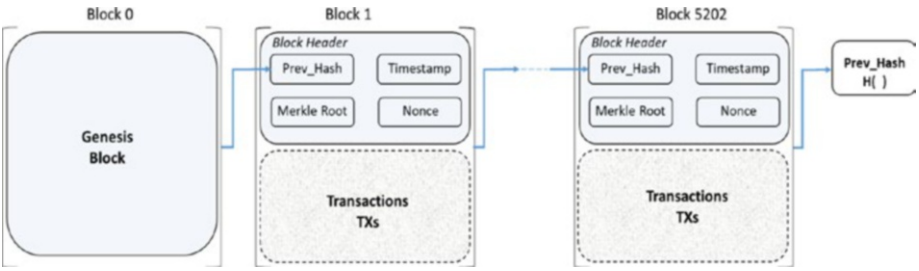


Figure 3-3. *The Bitcoin blockchain*

As you can see in this blockchain, there is a block header part that contains the header information and there is a body part where the transactions are bundled in every block. Every block's header contains the hash of the previous block. So, any change in any block in the chain will not be so easy; all the subsequent blocks have to be changed accordingly. Example: If someone tries to change a previous transaction that was captured in, say, block number 441, after changing the transaction,

the hash of this block that is in the header of block number 442 will not match, so it has to be changed as well. Changing the header with the new hash will then require you to update the hash in the block header of the next block in the sequence, which is block number 443, and this will go on till the current block and this is tough work to do. It beccomes almost impossible when we know that every node has it's own copy and hacking into all the nodes, or at least 51% of them, is infeasible.

In the blockchain, there is only one true path to the genesis block. However, if you start from the genesis block, then there can be forks. When two blocks are proposed at the same time and both of them are valid, only one of them would become a part of the true chain and the other gets orphaned. Every node builds on the longest chain, and whichever it hears first and whichever becomes the longest chain will be the one to build on. Such a scenario can be represented as shown in Figure 3-4.

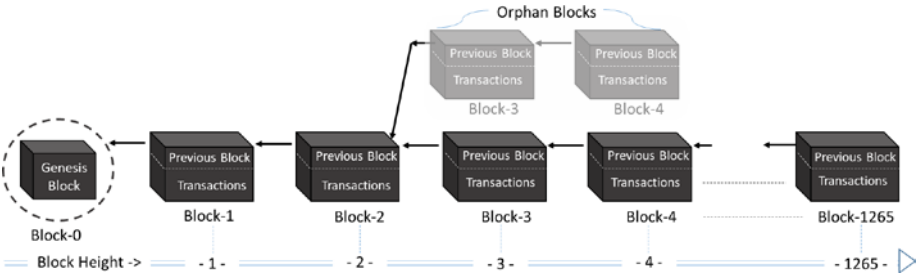


Figure 3-4. *Orphan blocks in true blockchain*

Observe in Figure 3-4 that at block height-3, two blocks are proposed to become block-3, but only one of them could make it to the final blockchain and the rest got orphaned out. It is evident that at a certain block height, there is a possibility of one or more blocks because there can as well be some orphaned blocks at this height, so block height is not the best way to uniquely identify a block and block hash is the right way to do so.

Block Structure

The block structure of a Bitcoin blockchain is fixed for all blocks and has specific fields with their corresponding required data. Take a look at Figure 3-5, a birds-eye view of the entire block structure, and then we will learn more about the individual fields later in this chapter.

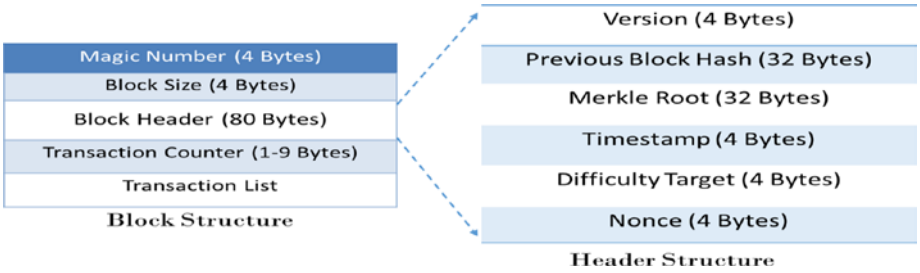


Figure 3-5. *Block structure of Bitcoin blockchain*

A typical block structure appears as shown in Table 3-1.

Table 3-1. *Block Structure*

Field	Size	Description
Magic Number	4 bytes	It has a fixed value 0xD9B4BEF9, which indicates the start of the block and also that the block is from the mainnet or the production network.
Block Size	4 bytes	This indicates the size of the block. The original Bitcoin blocks are of 1MB and there is a newer version of Bitcoin called “Bitcoin Cash” whose block size is 2MB.
Block Header	80 bytes	It comprises much information such as Previous Block’s hash, Nonce, Merkle Root, and many more.

(*continued*)

Table 3-1. (*continued*)

Field	Size	Description
Transaction Counter	1–9 bytes (variable length)	It indicates total number of transactions that are included within the block. Not every transaction is of the same size, and there is a variable number of transactions in every block.
Transaction List	Variable in number but fixed in size	It lists all the transactions that are taking place in a given block. Depending on block size (whether 1MB or 2MB), this field occupies the remaining space in a block.

Let us now zoom in (Table 3-2) to the "Block Header" section of the blocks and learn the various different fields that it maintains.

Table 3-2. *Block Header Components*

Field	Size	Description
Version	4 bytes	It indicates the version number of Bitcoin protocol. Ideally each node running Bitcoin protocol should have the same version number.
Previous Block Hash	32 bytes	It contains the hash of the block header of the previous block in the chain. When all the fields in the previous block header are combined and hashed with SHA256 algorithm, it produces a 256-bit result, which is 32 bytes.

(*continued*)

Table 3-2. (*continued*)

Field	Size	Description
Merkle Root	32 bytes	Hashes of the transactions in a block form a Merkle tree by design, and Merkle root is the root hash of this Merkle tree. If a transaction is modified in the block, then it won't match with the Merkle root when computed. This way it ensures that keeping the hash of the previous block's header is enough to maintain the secured blockchain. Also, Merkle trees help determine if a transaction was a part of the block in O(*n*) time, and are quite fast!
Timestamp	4 bytes	There is no notion of a global time in the Bitcoin network. So, this field indicates an approximate time of block creation in Unix time format.
Difficulty Target	4 bytes	The proof-of-work (PoW) difficulty level that was set for this block when it was mined
Nonce	4 bytes	This is the random number that satisfied the PoW puzzle during mining.

The block fields and their corresponding explanations as presented in the previous tables are good enough to start with, and we will explore more of only a few fields that require a more detailed explanation.

Merkle Tree

We have covered the concept of Merkle trees in the previous chapter. In this section, we will just take a look at how Bitcoin uses Merkle trees. Each block in a Bitcoin blockchain contains the hash of all the transactions, and the Merkle root of all these transactions is included in the header of

that block. In a true sense, when we say that each block header contains the hash of the entire previous block, the trust is that it just contains the hash of the previous block's header. Nonetheless, it is enough, because the header already contains the Merkle root. If a transaction in the block is altered, the Merkle root will not match anymore and such a design still preserves the integrity of the blockchain.

The Merkle tree is a tree data structure of the hash of the transactions. The "Leaf Nodes" in the Merkle tree actually represent the hash of the transactions, whereas the root of the tree is the Merkle root. Refer to Figure 3-6.

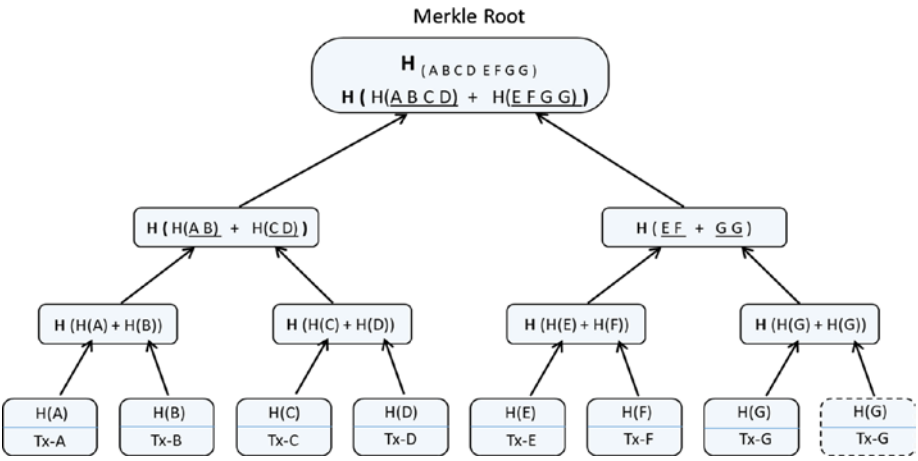


Figure 3-6. *Merkle-tree representation*

Notice that the hash of the seven transactions A, B, C, D, E, F, and G form the leaf of the tree. Since there are seven transactions but the total leaf nodes should be even in a binary tree, the last leaf node gets repeated. Each transaction hash of 32 bytes (i.e., 256 bits) is calculated by applying SHA256 twice to the transactions. Similarly, the hash of two transactions are concatenated (62 bytes) and then hashed twice with SHA256 to get the parent hash of 32 bytes.

Only the Merkle path to a transaction is enough to verify if a transaction was a part of any block and is quite efficient therefore. So, the actual blockchain can be represented as shown in Figure 3-7.

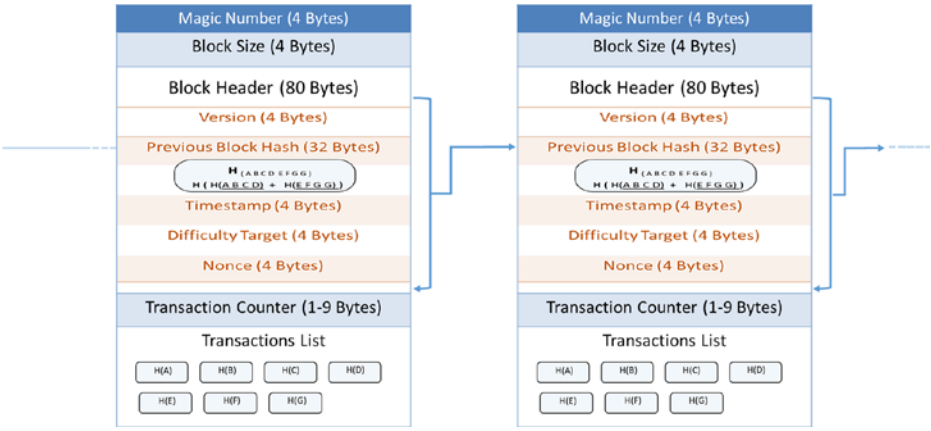


Figure 3-7. *Merkle tree representation*

Difficulty Target

The difficulty target is the one that drives the PoW in Bitcoin. The idea is that once a block is filled with valid transactions, the hash of that block's header needs to be calculated to be less than the difficulty target in the same header. The nonce in the header starts from zero. The miner has to keep on incrementing this nonce and hashing the header till the hash value is less than the target.

The difficulty bits of four bytes (32 bits) in the headers define what would be the *target* value (256 bits) for that block to be mined. The nonce should be found such that the hash of the entire header should be less than the target value. Remember that the lower the target value, the more difficult it would be to find a header hash that would be less than the target. Since Bitcoin uses SHA256, every time you hash a block header, the output is any number between 0 and 2^{256}, which is quite a big number.

If with your nonce the hash is less than the target, the block will be accepted by the entire network, else you have to try with a different nonce till it satisfies the condition. At this point, it is still not clear how the difficulty target is calculated with the difficulty bits in each header.

The target can be derived from the four-byte (8 hexadecimal numbers) difficulty bits in the header using a predefined formula that every node has by default, as it came along with the binaries during installation. Following is the formula to compute the difficulty:

$$target = coefficient * 2^{(8 * (exponent - 3))}$$

Notice that there is a "coefficient" and there is also an "exponent" term in this formula, which are present as a part of the four-byte difficulty bits. Let us take an example to explain this better. If the four-byte difficulty bits in hex form are 0x1b0404cb, then the first two hex digits form the exponent term, which is (0x1b), and the remaining form the coefficient term, which is (0x0404cb) in this case. Solving for the target formula with these values:

$$target = 0x0404cb * 2^{(0x08 * (0x1b - 0x03))}$$
$$target = 0x00000000000404$$
$$CB0000000000000000000000000000000000000000000000$$

Bitcoin is designed such that every 2,016 blocks should take two weeks to be generated and if you do the math, it would be around ten minutes for every block. In this asynchronous network, it is difficult to program like this where every block takes exactly ten minutes with the kind of PoW mechanism in place. In reality, it is the average time for a block, and there is a possibility that a Bitcoin block gets generated within a minute or it may very well take 15 minutes to be generated. So, the difficulty is designed to increase or decrease depending on whether it took less or more than two weeks to find 2,016 blocks. This time taken for 2,016 blocks can be found using the time present in the timestamp fields of every block header. If it took, say, **T** amount of time for 2,016 blocks, which is never exactly

two weeks, the difficulty target in every block is multiplied by (**T / 2 weeks**). So, the result of [difficulty target × (**T / 2 weeks**)] will be increased if **T** was less and decreased otherwise.

It is evident by now that the difficulty target is adjustable; it could be set more difficult or easier depending on the situation we explained before. You must be wondering, who adjusts this difficulty when the system is decentralized? One rule of thumb that you must always keep in mind is that whatever happens in such a decentralized design has to happen individually at every node. After every 2,016 blocks, all the nodes individually compute the new difficulty target value and they all conclude on the same one because there is already a formula defined for it. To have this formula handy, here it is once again:

```
    New Target= Old Target * (T / 2 weeks)
⇨   New Target= Old Target * (Time taken for 2016 Blocks in
    Seconds / 12,09,600 seconds)
```

Note The parameters such as 2,016 blocks and TargetTimespan of two weeks (12,09,600 seconds) are defined in **chainparams.cpp** as shown following:

```
consensus.nPowTargetTimespan = 14 * 24 * 60 * 60; // two weeks
consensus.nPowTargetSpacing = 10 * 60;
consensus.nMinerConfirmationWindow = 2016; //
nPowTargetTimespan / nPowTargetSpacing
```

Note here that it is (T / 2 weeks) and not (2 weeks / T). The idea is to decrease the difficulty target when it is required to increase the complexity, so it takes more time. The lesser the target hash, the more difficult it gets to find a hash that is less than this target hash. Example: If it took ten days to mine 2,016 blocks, then (T / 2 weeks) would be a fraction, which when

multiplied by "Old Target" further reduces it and "New Target" would be a value less than the old one. This would make it difficult to find a hash and would require more time. This is how the time between blocks is maintained at ten minutes on average. Imagine that the difficulty target was fixed and not adjustable; what do you think the problem would be? Remember that the computation power of the hardware increases with time as more powerful computers are introduced for block mining. A situation where 10s or 100s or even 1,000s of blocks are proposed at the same time is not desirable for the network to function properly. So, the idea is that, even when more and more powerful computing nodes enter into the Bitcoin network, avrage time required to propose a block should still be ten minutes by adjusting the difficulty target. Also, a miner's chances of proposing a block depends on how much hash power they have compared with the global hash power of all miners included.

Are you thinking why ten minutes, and why not 12 minutes? Or why not six minutes? Just keep in mind that there has to be some time gap for all the nodes in a decentralized asynchronous system to agree on it. If there was no time gap, so many blocks would arrive with just fractional delays and there wouldn't be any optimization benefit of **blockchain** as compared with **transaction chain**. Every transaction is a broadcast and every new block is also a broadcast. Also, the orderliness that a blockchain brings to the system is quite infeasible by the transaction chain. With the concept of blocks, it is possible to include the unrelated transactions from any sender to any receiver in blocks, which is not easy to maintain with the transaction chain. One valid block broadcast is more efficient compared with individual transaction broadcast after validation. Now, coming back to the discussion of ten minutes, it can very well be a little less or a little more but there should certainly be some gap between two consecutive blocks. Imagine that you are a miner and mining block number 4567, but some other miner got lucky and proposed block number 4567, which you just received while solving the cryptographic puzzle. What you would do now is validate this block and if it is valid, add it to your local copy of the

blockchain and immediately start on mining the 4568. You wouldn't want someone else to propose 4568 already while you just finished validating block 4567, which you received a little later compared with other miners due to network latency. Now the question is: *is this 10 minutes the best possible option*? Well, it is difficult to explain this in one word, but a ten-minute gap addresses a lot of issues due to an asynchronous network, time delays, packet drops, system capacity, and more. There is a possibility that it could be optimized further to, say, five minutes or so, which you can see in many new cryptocurrencies and other blockchain use cases.

The Genesis Block

The very first block as you can see in the following code, the block-0, is called the *genesis* block. Remember that the genesis block has to be hardcoded into the blockchain applications and so is the case with Bitcoin. You can consider it as a special block because it does not contain any reference to the previous blocks. The Bitcoin's genesis block was created in 2009 when it was launched. If you open the Bitcoin Core, specifically the file *chainparams.cpp*, you will see how the genesis block is statically encoded. Using a command line reference to Bitcoin Core, you can get the same information by querying with the hash of the genesis block as shown below:

```
$ bitcoin-cli getblock 000000000019d6689c085ae165831e934
ff763ae46a2a6c172b3f1b60a8ce26f
```

Output of the preceding command:

```
{
    "hash" : "000000000019d6689c085ae165831e934ff763ae46a2a6
    c172b3f1b60a8ce26f",
    "confirmations" : 308321,
    "size" : 285,
    "height" : 0,
```

```
    "version" : 1,
    "merkleroot" : "4a5e1e4baab89f3a32518a88c31bc87f618
    f76673e2cc77ab2127b7afdeda33b",
    "tx" : ["4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc
    77ab2127b7afdeda33b"],
    "time" : 1231006505,
    "nonce" : 2083236893,
    "bits" : "1d00ffff",
    "difficulty" : 1.00000000,
    "nextblockhash" : "00000000839a8e6886ab5951d76
    f411475428afc90947ee320161bbf18eb6048"
}
```


If you convert the Unix time stamp as shown in the previous output, you will find this date-time information: *Saturday 3rd January 2009 11:45:05 PM.* You can as well get the same information from the website `https://blockchain.info`. Just navigate to this site and paste the hash value in the right top search box and hit "Enter." Here is what you will find (Table 3-3)

Table 3-3. *Transaction Information*

Summary	
Number Of Transactions	1
Output Total	50 BTC
Estimated Transaction Volume	0 BTC
Transaction Fees	0 BTC
Height	0 (Main Chain)
Timestamp	2009-01-03 18:15:05
Received Time	2009-01-03 18:15:05

(*continued*)

Table 3-3. (*continued*)

Summary	
Relayed By	Unknown
Difficulty	1
Bits	486604799
Size	0.285 kB
Weight	0.896 kWU
Version	1
Nonce	2083236893
Block Reward	50 BTC

Table 3-4. *Hash Information*

Hashes	
Hash	000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
Previous Block	0000000000000000000000000000000000000000000000000000000000000000
Next Block(s)	00000000839a8e6886ab5951d76f411475428afc90947ee320161bbf18eb6048
Merkle Root	4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b

In this Block-0, there is just one transaction, which is a coinbase transaction. Coinbase transactions are the ones that the miners get. There are no inputs to such transactions and they can only generate new Bitcoins. If you explored the transactions associated in this block, here is how it would look (Figure 3-8).



Figure 3-8. *Coinbase transaction in Block-0*

The Bitcoin Network

The Bitcoin network is a peer-to-peer network, as discussed already. There is no centralized server in such a system and every node is treated equally. There are no master–slave phenomena and no hierarchy as well in such a system. Since this runs on the Internet itself, it uses the same TCP/IP protocol stack as shown in Figure 3-9.

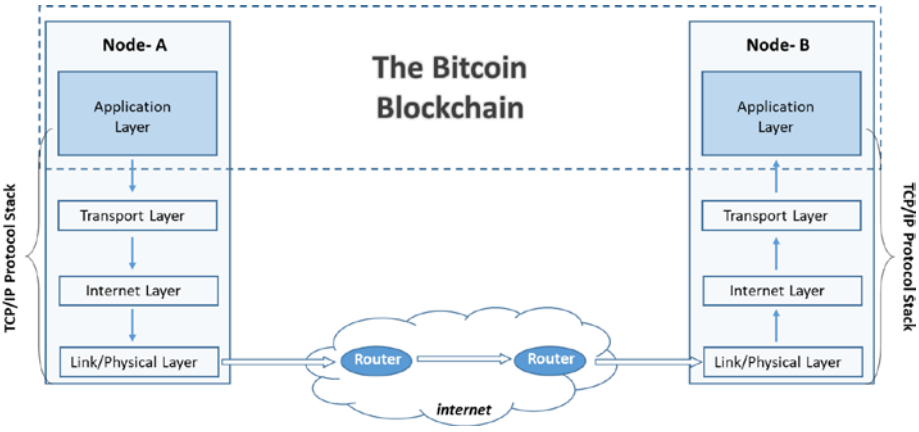


Figure 3-9. *The Bitcoin blockchain network on the Internet*

The above diagram shows how Bitcoin networks coexist on the same Internet stack. The Bitcoin network is quite dynamic in the sense that nodes can join and leave the netwrk at will and the system still works. Also, despite being asynchronous in nature and with network delays and packet drops, the system is very robust–thanks to the design of Bitcoin!

The Bitcoin network is a decentralized network with no central point of failure and as well no central authority. With such a design, how would you assess how big the Bitcoin network is? There is no proper way of estimating this as the nodes can join and leave at will. However, there are some attempts at researching the Bitcoin network, and some claim that there are close to 10,000 nodes that are mostly connected to the network all the time and there can be millions of nodes at a time.

Every node in the Bitcoin network is equal in terms of authority and has a flat structure, but the nodes can be full nodes or lightweight nodes. The full nodes can do almost every permissible activity in the Bitcoin system, such as mining transactions and broadcasting transactions, and can provide wallet services. The full nodes also provide the routing function to participate in and maintain the Bitcoin network. To become a full node, you have to download the entire blockchain database that containss the entire transactions taken place till now. Also, the node must stay permanently connected to the Bitcoin network and hear all transactions taking place. It is important that you have a good network connection, good storage (at least 200GB), and at least 2GB RAM dedicated to it. This requirement may further change and require more resources with time.

On the other hand, lightweight nodes cannot mine new blocks but can verify transactions by using Simplified Payment Verification (SPV). They are otherwse termed "thin clients." A vast majority of nodes in the Bitcoin network are SPVs. They can as well participate in pool mining where there are many nodes trying to mine new blocks together. Lightweight nodes can help verify the transactions for the full nodes. A good example of an SPV is a wallet (the client). If you are running a wallet and someone sends money to you, you can act as a node in the Bitcoin network and download the relevant transactions to the one made to you so you can check if the person sending you Bitcoins actually owned them.

It is important to note that an SPV is not as secured as a fully validating node because it usually contains the block headers and not the entire blocks. As a result, SPVs cannot validate transactions since they don't have them for a block and also because they do not have all the unspent transaction outputs (UTXOs) except for their own.

Network Discovery for a New Node

Now think about, when a new node wants to join the network, how would it contact the network? It is not an intranet with a 192.168.1.X network where you can broadcast to the IP 192.168.1.255 so that whichever computer is a part of the 192.168.1.X network gets the broadcast message. The network switches are designed to allow such broadcast packets. However, remember that we are talking about the Internet, which Bitcoin is sitting on. If you are running a node in London, there is a possibility that there are other nodes in London, Russia, Ireland, the United States, and India and all of them are connected through the Internet with some public facing IP address.

The question here is that when a fresh node joins the network, how does it figure out the peer nodes? There is no central server somewhere to respond to their request the way a typical Internet-based web application works. Blockchain is decentralized, remember? When started for the first time, a Bitcoin Core or BitcoinJ program does not have the IP address of any full node. So, they are equipped with several methods to find the peers. One of them is DNS seeds. Several DNS seeds are hardcoded in them. Also, several host names are maintained in the DNS system that resolve to a list of IP addresses that are running the Bitcoin nodes. DNS seeds are maintained by Bitcoin community members. Some community members provide static DNS seeds by manually entering the IP addresses and port numbers. Also, some community members provide dynamic DNS seed servers that can automatically get the IP addresses of active Bitcoin nodes that are running on default Bitcoin ports (8333 for *mainnet* and 18333

for *testnet*). If you perform NSLOOKUPs on the DNS seeds, you will get a bunch of IP addresses running Bitcoin nodes.

The clients (Bitcoin Core or BitcoinJ) also maintain a hardcoded list of IP addresses that point to some (not one!) stable Bitcoin nodes. Such nodes can be called bootstrap nodes whose endpoints are already available with the source code itself. Every time one downloads the binaries, a fresh list of active nodes get downloaded along with the binaries. Once a Bitcoin node connection is established, it is very easy to pull the list of other Bitcoin nodes active at that point in time. A pictorial representation of how a new node becomes a part of the network can be found in the following figures.

Step-1:

Imagine that there were six nodes active at some point in time in the Bitcoin network. Refer to Figure 3-10.

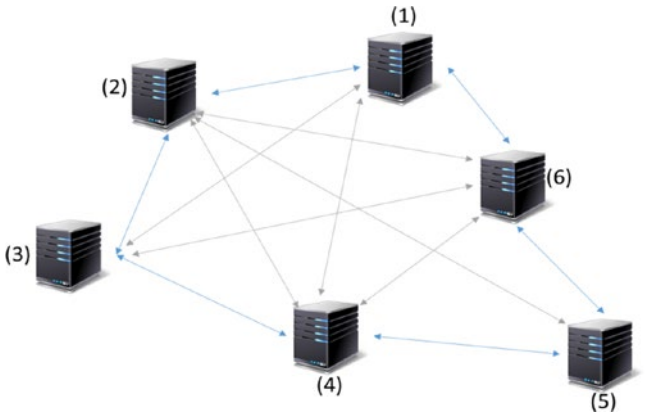


Figure 3-10. *Bitcoin network in general*

Step-2:

There is a new node, say, a seventh node that just showed up and is trying to join the existing Bitcoin network, but does not have any connection yet. Refer to Figure 3-11.

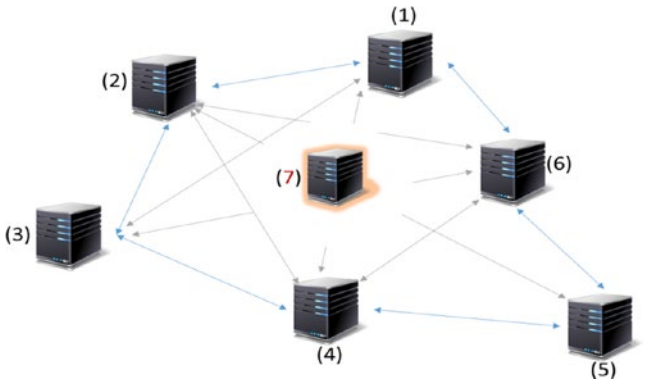


Figure 3-11. *A new node trying to join the network*

Step-3:

The seventh node will try to reach out to as many nodes as it can either using DNS seeds or using the list of stable Bitcoin nodes in the list that it has–as shown in Figure 3-12.

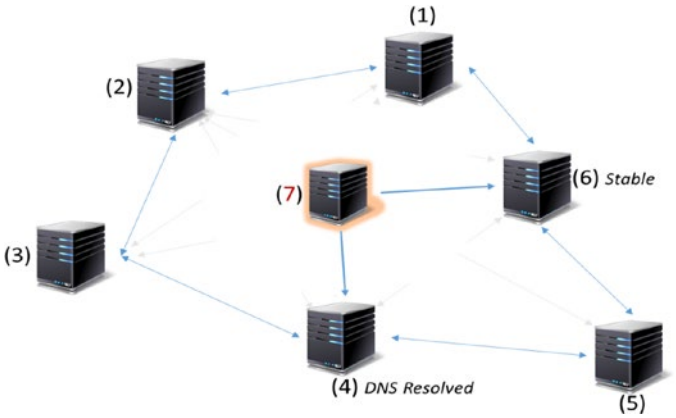


Figure 3-12. *New Bitcoin node contacts some peers*

In the diagram, we have skipped the DNS resolution part. It is the same as when you browse any website with its name and post DNS resolution the IP address is retrieved, which is then used as the destination

webserver's address to send TCP packets to. To connect to a new peer, the node establishes a TCP connection on port 8333 (port 8333 is well known for Bitcoins but could be different). Then the two nodes handshake with information such as version number, time, IP addresses, height of blockchain, etc. The actual Bitcoin code for "Version" message defined in *net.cpp* is as shown in the following:

```
PushMessage( "version", PROTOCOL_VERSION, nLocalServices,
nTime, addrYou, addrMe,
                nLocalHostNonce, FormatSubVersion(CLIENT_NAME,
                CLIENT_VERSION,
                std::vector<string>()), nBestHeight, true );
```

Through this Version message, the compatibility between the two nodes is checked as the first step toward further communication.

Step-4:

In the fourth step, the requested nodes will respond with the list of IP addresses and corresponding port numbers of the other active Bitcoin nodes that they are aware of. Please note that it is possible for some of active nodes to not be aware of each and every Bitcoin node in the network at any time. The port number is important because once the TCP packets reach the destination node, it is the port number that is used by the operating system to direct the message to the correct application/process running on the system. Please refer to Figure 3-13.

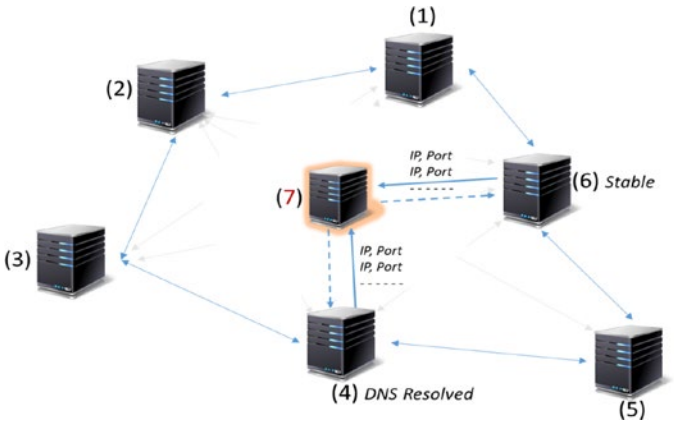


Figure 3-13. *Peer Bitcoin nodes respond to the network request by a new node*

Note that, only one peer may be enough to bootstrap the connection of a node to the Bitcoin network; the node must continue to discover and connect to new peers. This is because nodes come and go at will and no connection is reliable.

Step-5:

In the fifth step, the new seventh node establishes connection with all the reachable Bitcoin nodes, depending on the list it received from the nodes contacted in the previous step. Figure 3-14 represents this.

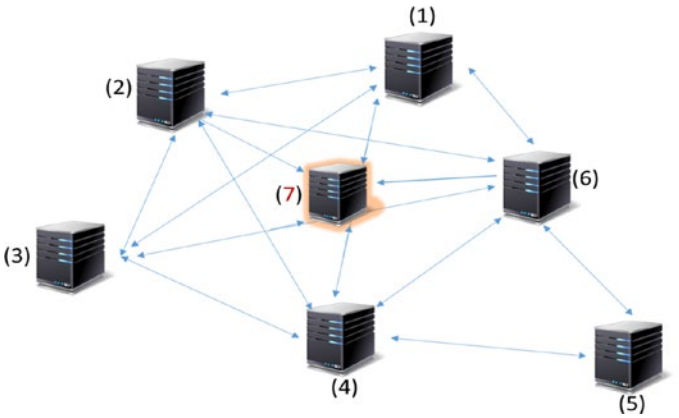


Figure 3-14. *A new node becomes a part of the Bitcoin network*

Bitcoin Transactions

Bitcoin transactions are the fundamental building blocks of the Bitcoin system. There are basically two broader categories of Bitcoin transactions:

- **Coinbase transaction**: Every block in Bitcoin blockchain contains one coinbase transaction included by the miners themselves to be able to mine new coins. They do not have control of how many coins they can mine in every block because it is controlled by the network itself. It started with 50 BTC in the beginning and keeps halving till it reaches 21 Million Bitcoins in total.
- **Regular transactions**: The regular transactions are very similar to currency exchanges in general, where one is trying to transact some amount of money that they own with another. Typically, in Bitcoin, everything is present as transactions. To spend some amount, one has to consume previous transaction(s) where they

received that amount—these are regular transactions in Bitcoin. Our main focus in this chapter will be on these regular transactions.

Each owner of a Bitcoin can transfer the coin to someone else by digitally signing a hash of the previous transaction where they had received the Bitcoin along with the public key of the recipient. The payee or the recipient already has the public key of the payer so they can verify the transaction. The following figure (Figure 3-15) is from the white paper of Satoshi Nakamoto that pictorially demonstrates how it works.

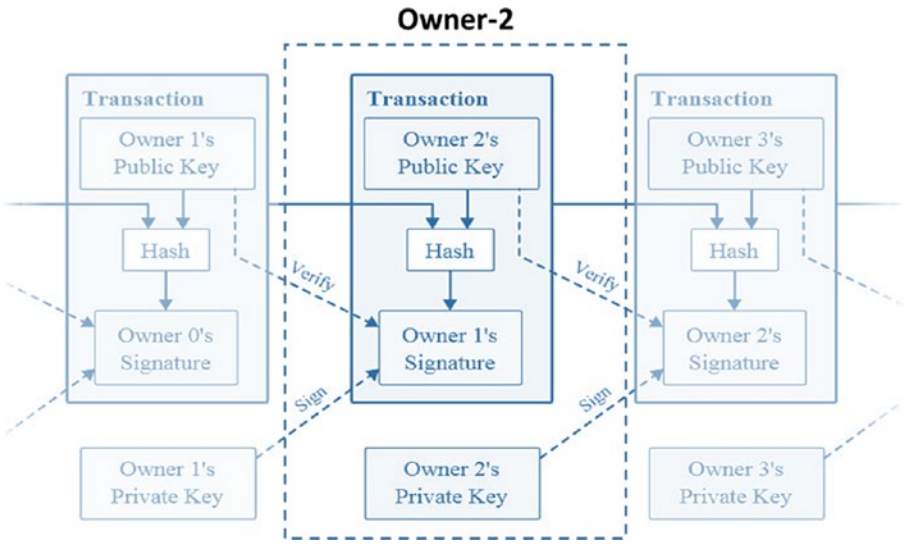


Figure 3-15. *Bitcoin transaction*

Notice only the highlighted Owner-2 section in the diagram. Since Owner-1 is initiating this transaction, he is using his private key for signing the hash of two items: one is the the previous transaction where he himself received the amount and the second is Owner-2's public key. This signature can be easily verified using the public key of Owner-1 to ensure that it is a legitimate transaction. Similarly, when Owner-2 will initiate

a transfer to Owner-3, he will use his private key to sign the hash of the previous transaction (the one he received from Owner-1) along with the public key of Owner-3. Such a transaction can be, and will be, verified by anyone who is a part of the network. Obviously because every transaction is broadcast, most of the nodes will have the entire history of transactions to be able to prevent double-spend attempts.

There is no principle of closing balance in a Bitcoin network, and the total amount one holds is the summation of all incoming transactions to the public addresses you own. You can create as many public addresses as you want. If you have ten public addresses, then whatever transactions were made to that public address, you can spend those transactions (unspent transactions or UTXOs) using your private key. If you have to spend, say, five Bitcoins, you have a coupe of choices:

- Use one of the previous transactions where you received five or more Bitcoins. Transfer five Bitcoins to the recepient, some amount as transaction fee and the remainder to yourself. Refer to Figure 3-16.

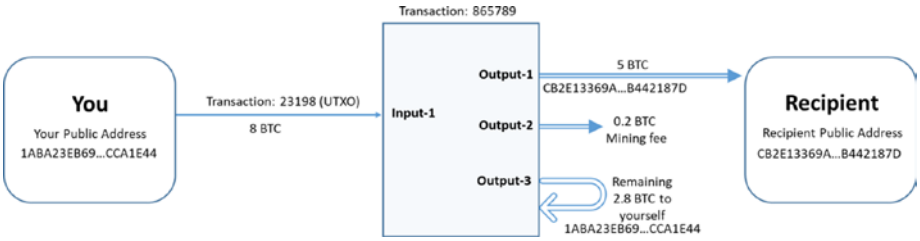


Figure 3-16. *Bitcoin transaction with one transaction input*

- Use multiple previous transactions that you had received that would sum up to more than five Bitcoins. Transfer five Bitcoins to the recepient, some amount as transaction fee and the remainder to yourself. Refer to Figure 3-17.

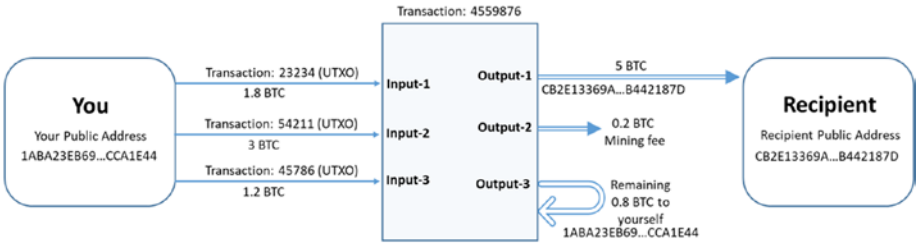


Figure 3-17. *Bitcoin transaction with multiple transactions input*

As you can see, every transaction takes as input the previous transaction(s). There is no account maintained that says you have eight BTC, and you can spend anything below this amount; if you spend five BTC, the remaining balance would be three BTC. In Bitcoin, everything is a transaction where there are inputs and outputs. If the outputs are not spent yet, they are the UTXOs.

We are aware that every transaction in the network is broadcast to the entire network. Whether someone is maintaining a node or not, they can still make a transaction and that transaction is published to all the accessible Bitcoin nodes. The receiver Bitcoin nodes then further broadcast the transactions to other nodes and the entire network is usually flodded with transactions. This is sometimes referred to as the gossip protocol and plays an important role in preventing double-spend attacks. Recollect from Chapter 2 that the only way to prevent double-spend is to be aware of all transactions.

Each node maintains a set of all the transactions that they hear about and broadcasts only the new ones, which were not a part of the list already. Nodes maintain the transactions in the list till the time the transaction gets into a block and is a part of the blockchain. This is because there is a chance that even if a block has all valid transactions and is proposed as a valid block, it can still get orphaned by not being a part of the longest chain. Once it is confirmed that the block is now a part of the longest chain, the transactions that are there in that block are taken off from the

list of transactions. Each full node in the Bitcoin network must maintain the entire list of unspent transactions (UTXOs) even though they are in the millions. If a transaction is in the list of UTXOs, then it may not be a double-spend attempt. Upon confirming a transaction is not a double-spend attack and also validating the transactions from other perspectives, a node broadcasts such transactions. If you are wondering how fast it would be to search millions of UTXOs to check for double-spend, you are on track. Since the transaction outputs are ordered by their hashes, searching for an item in an ordered hash list is quite fast.

Let us now think and dig deeper into a double-spend scenario. It is very possible that Alice (A) tries to pay Bob (B) and Charlie (C) the same transaction (input to a transaction is a previous transaction and there is no concept of closing balance). Such a scenario would appear as shown in Figure 3-18.

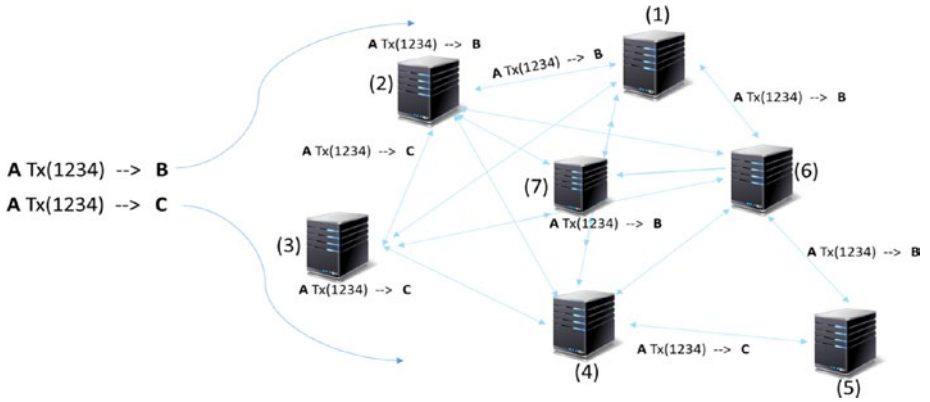


Figure 3-18. *A double-spend transaction scenario in Bitcoin network*

Notice in the figure the following scenarios:

- **A** is trying to spend the same transaction to **B** and **C**.
- Node-2 received the transaction **A** Tx(1234) --> **B** and Node-3 received the transaction **A** Tx(1234) --> **C**.

- For Node-2 and Node-3, their respective transactions received were legitimate transactions.
- When Node-3 tries to broadcast the transaction **A** Tx(1234) --> **C** to Node-2 (every node broadcasts new transactions), Node-2 would refuse this transaction because it already has a transaction **A** Tx(1234) --> **B** with the same input transaction Tx(1234).
- Similar things happen with other nodes as well, and they may have either the transaction "**A** Tx(1234) --> **B**" or "**A** Tx(1234) --> **C**", whichever reached them faster, but not both.
- During mining, whichever node gets to propose the block will include the transaction it has. This transaction would be a part of the blockchain and the rest of the nodes that are holding the other transaction would simply drop the transaction with Tx(1234) because it will no longer be a UTXO.

Consensus and Block Mining

In the previous section we looked at granular transactions. We will learn how the transactions are bundled together to form a block and a consensus is achieved among nodes, so the entire network accepts that block to extend the blockchain. Note that "block mining" refers to successfully creating a new block in the blockchain. In Bitcoin, it is the distributed PoW consensus algorithm that helps mine new blocks by maintaining decentralization. Achieving distributed consensus in such a network is very difficult. Though it has been around for decades for distributed systems such as Facebook, Google, Amazon, and many more because they have millions of servers that require consistency in the

data they store, the term *consensus* is very much popularized because of Bitcoins. We will get into the nuts and bolts of consensus and mining in this chapter.

First, just keep in mind that everything in a Bitcoin network is represented as transactions. If you want to make a transaction, you have to consume one or more previous transactions as input and make another transaction. We already know that one has to sign a transaction using their private key to ensure that the right person is making the transaction. Despite such cryptographic security, can't that person sign a transaction that they already spent? Example: Alice received ten Bitcoins in a transaction with transaction number 1234. She can very well spend the same transaction 1234 and give away those ten Bitcoins to Bob and Charlie. Since she will sign with her private key, which means it is an authentic transaction, what do you think can prevent her from double-spending? Please note that there is no way in Bitcoin that you can prevent her from attempting to make a double-spend, but the system is designed so that such an attempt will not be successful. The only way to prevent such attempts is to be aware of all the transactions that are taing place. This is why all transactions in Bitcoin are broadcast to the entire network. Once a transaction is spent, it is no longer a part of the UTXOs and a new transaction number is generated to be a part of the UTXO, which only the recipient can spend. This is the way nodes can validate transactions.

Also, the only way you can prevent a double-spend attack is by knowing all the transactions. When you are aware of all transactions, you would know the spend and the UTXOs. When a new block is proposed by a miner, it is required that all the transactions in the block are valid. Does it mean that the node proposing a block cannot include a invalid transaction? The answer is "Yes." They can certainly inject a fraudulent transaction but the rest of the nodes will reject it. The PoW that the node would have done (we will get into the details shortly) by spending computer resources and electricity would be in vain! So, a node would never want to propose an invalid block, thanks to the PoW consensus.

Despite not having a notion of global time, observe that the transactions are clubbed together to form a block that becomes a part of blockchain, more blocks get added to the chain one by one, and there is an order! Note carefully that the order in which the transactions took place is preserved by the blockchain. This way, consensus happens at block level, which propagates all the way to granular transactions.

Based on what we have understood so far, now we know that every node in the Bitcoin network has its own copy of blockchain, and there is no "global blockchain" as such; it is a decentralized network after all. Each node in all those copies of blockchains comprises many transactions. It is also true that every node maintains a list of UTXOs and when given a chance (A node is randomly selected–we will see how) to propose a block, they include as many transactions as possible up to the block limit of 1MB or 2MB. If the block successfully makes it to the blockchain, they are removed from the list of UTXOs. Note here that every node may have different outstanding transactions lists because there is a possibility that some transactions are not heard by some of the nodes.

It's time now to see how the PoW algorithm really works. We learned about the difficulty target field in the header of each block. Every mining node tries to solve the cryptographic puzzle with an expectation to get lucky and propose a block. The reason they are so desperate in proposing a block is because they get great benefits when their proposed block becomes a part of the blockchain. Every transaction that an individual makes, they can set aside some transaction fee for the miners. We know that all nodes maintain the list of transactions that are not yet a part of the blockchain and when they get a chance to propose a block, they take as many transactions as they can and form a block. It is obvious that they will take all those transactions that would give them the highest profit and leave the ones with minimum or no transaction fees. It may take some time for the transactions with low transaction fee to get into the blocks, and

chances are less for the ones with no transaction fee at all. Apart from the transaction fee, the nodes that propose a new block are rewarded with new Bitcoins. With every successful block, new Bitcoins get generated and the miner who proposed the block gets all of those; this is the only way new Bitcoins get created in the Bitcoin system. This is called "block reward." Technically, the node that proposes a block includes a special transaction called "coin creation" in the proposed block where the recipient address is the one that the miner owns. When the Bitcoin was first launched, the block reward was 50 Bitcoins (BTC). By design, there can only be 21,000,000 BTCs in total, so the block reward gets halved every 210,000 blocks. It started at 50, then it became 25, then 12.5, and it goes on this way till at some point in time (when it reaches 21,000,000 BTCs) it trends to zero. Following is the code snippet from Bitcoin Core (*main.cpp*) that shows this halving process:

```
int64_t GetBlockValue(int nHeight, int64_t nFees)
{
    int64_t nSubsidy = 50 * COIN;
    int halvings = nHeight / Params().SubsidyHalvingInterval();

    // Force block reward to zero when right shift is undefined.
    if (halvings >= 64)
        return nFees;

    // Subsidy is cut in half every 210,000 blocks which will
    occur approximately every 4 years.
    nSubsidy >>= halvings;

    return nSubsidy + nFees;
}
```

Notice in the previous code snippet how the block reward gets halved. The following explanation gives a better picture of this design:

```
//Block reward reduced by half, remains at 50%
BlockReward = BlockReward >> 1;

//Block reward further reduced by half, remains at 25%
BlockReward = BlockReward-(BlockReward>>2);

//Block reward further reduced by half, remains at 12.5%
BlockReward = BlockReward - (BlockReward>>3);
```

Even though the rewards look lucrative, it is not so easy to get lucky and be the node that gets to propose a block. If you are not lucky enough, all the work you did would be in vain; that's a disadvantage! So, what is it that the nodes do as a PoW? Let's get back to the *difficulty* puzzle now. Every mining node at all times is working to propose a block, but only one succeeds at a given point in time. Assume that a block is proposed already, and now all mining nodes are working to propose a new block. Let us go through the process step by step and understand the whole flow:

Step-1:

The miners use a software to keep track of the transactions, eliminate the ones that already made it to a successful block in blockchain, reject the fraudulent transactions, and solve the cryptographic puzzle to propose a new block and relay that to the entire network. The best software to mine is the official Bitcoin Core but there have been many other variants that people have come up with. If you log into this link (https://bitcoin.org/en/download) you will find that the official Bitcoin Core is supported in Windows, Linux, Mac OS, Ubuntu, and ARM Linux. So, when we say that the mining node selects all the transactions (maybe the ones that give the miner the highest profit) till the block limit of 1MB (2MB for Bitcoin Cash), they also hash those transactions and generate the Merkle root that would become a part of this new block's header. This Merkle root represents all the transactions.

Step-2:

They prepare the block header. Apart from the nonce, the rest is all available at this step. It is the work of the mining node to find the nonce by hashing the block header twice and comparing it against the difficulty target to see if it is less than that. They keep changing the nonce till it satisfies this condition, and there is no shortcut to find a nonce quickly; one must try out every possible option. We already looked at how to compute the difficult target using the four bytes of data present in the header itself, and we learned how it changes every two weeks. See the following for how this process looks:

```
H [ H (Version | Previous Block Hash | Merkle Root | Time Stamp
| Difficulty Target |  Nonce) ]

< [ Difficulty Target ]
```

Step-3:

The miner keeps on changing the nonce field in step-2, by incrementing it by "1" till it satisfies the condition–it is a recursive phenomenon. The difficulty target for every node is the same and all of them are trying to solve the same problem, but remember that they may have different variants of transaction pools and hence the Merkle root for them would be different. Since every node is trying to extend the longest and main blockchain, so the previous block hash would be the same.

So, ultimately the Sha256 hash twice for the block header should be less than the target to be able to propose the block to the entire network. See the following example for a better understanding:

```
Target    : 0000000000000074cd0000000000000000000000000000000000
            0000000000000

Hash      : 0000000000000074cc4471deff052ced7f07347e4eda86c84
            5a2fcf0553ed7f0
```

Notice that the hash value and the target value have the same number of leading zeros (i.e., 14) and "74cc" is less than "74cd," so it satisfies the condition and this block can now be proposed. In many places, you would find that this explanation is simplified with ballpark values of both the *target* and the *hash*, and counting only the leading zeros. If the hash has more leading zeros than the target, then it satisfies the condition. Remember again that the more zeros in the target, the more difficult it gets in finding the hash that can satisfy the condition.

Let us connect this learning so far with the real Bitcoin implementation. We know that block creation time is set to ten minutes–it is coded up in Bitcoin binaries for 2,016 blocks in two weeks, as we discussed already, and does not change till a hard fork happens. You can browse blocks proposed and see the hashes that satisfied the difficulty target at the website `https://blockchain.info` and see for yourself that the hashes for different blocks would have different leading zeros, just to set the block creation time to ten minutes on average. In the initial days, the number of leading zeros was around nine or ten, and today it has increased to around 18 to 20 zeros. It may increase even further as and when more powerful computing nodes capable of more hash rates join the network.

Step-4:

Once a miner finds the valid block, they immediately publish the block to the entire network. Every node that receives this block individually checks again if the miner who proposed the block actually solved the mining puzzle. For these nodes to validate this, it is just one step, as shown below:

```
H [H (Version| PreviousBlock Hash | Merkle Root | Time Stamp |
Difficulty Target | Found Nonce)]

< [ Difficulty Target ]
```

Notice that they just use the block header that includes the nonce found by the proposing miner to see if the hash is less than the target and is valid. If it was a valid nonce and the condition satisfied, then they check for individual transactions proposed in the block with its Merkle root in the block header. If all transactions are valid, then they add this block to the local copy of their blockchain. This new block has the coinbase transaction that generates new coins (it started with 50 BTC, then 25, then 12.5, and keeps halving as discussed) as an award for the miner who proposed the valid block.

Note here that block mining is not an easy job, thanks to the PoW mining algorithm. For a node to propose an invalid block, it has to burn a lot of electricity and CPU cycles to find the nonce and propose the block that would ultimately get rejected by nodes in the network. Had it been an easy task, many nodes would just keep trying for it and flood the network with bad blocks. You must understand and appreciate by now how Bitcoin prevents such situations in a game theoretic way! It is always profitable for the miners to play by the rules and they do not gain any extra benefits by not following the rules.

In the previous steps, we learned the PoW mining procedure that is implemented in Bitcoins. One of the best things in this design is the random selection of a mining node that gets to propose a block. No one knows who would get lucky to find the right nonce and propose a block, it is purely a random phenomenon. We already know that generating a true random number is quite difficult and this is something that is the most vulnerable surface of attack for most cryptographic implementations. With such a design as Bitcoin's, random selection of a node to propose a block is truly random.

The next best thing in Bitcoin mining is the block reward. It is something that a miner who successfully proposes a new block gets, using the coinbase transaction in the same block. The miners also get the transaction fees associated with all the transactions they have included in

the block. So, mining reward for a block is a combination of block reward and transaction fee as shown below:

```
Mining Reward = Block Reward + Total transaction fees of all
transactions in the Block
```

We know that mining is the only way new Bitcoins are created in the Bitcoin system, but is that the purpose of mining? No! The purpose of mining is to mine new blocks, and generation of new Bitcoins and also the transaction fee is to incentivize the miners so that more and more miners are interested in mining. Obviously though, why would you mine if you are not making good money? This is again game theory. A proper incentivization mechanism is the key to make a sytem decentralized and self-sustainable. Notice that the Bitcoin system does not have a way to penalize nodes that do not play honestly, it only rewards honest behavior. Actors in the Bitcoin blockchain network such as individuals who just use Bitcoin or the miners are all identified using their public keys. It is possible for them to generate as many key pairs as possible and this makes it a psydonemous system. A node cannot be uniquely identified with its public key that it has used in the coinbase transaction, as in the very next moment it can create a new key pair and expose itself with a new network address. So, proper incentivization stands to be the best way to ensure the actors in the system play honestly–again the beauty of game theory!

Here is a question for you now. After a block was broadcast, let's say a node verified it, found the nonce and transactions and everything else to be valid, and included it in its local copy of blockchain. Does this mean that the transactions that were there in the block are all settled and confirmed now? Well, not really! There is a chance that two blocks came in at the same time and while one node started extending one of them, there is a chance that a majority of the nodes are extending on the other block. Eventually, the longest blockchain becomes the original chain. This is a scenario when a block that is absolutely valid, with all legitimate transactions and a proper nonce value that satisfied the mining puzzle, can still get abandoned by the Bitcoin network. Such blocks that do not

become a part of the final blockchain are called orphaned blocks. Now, this explanation indicates that there is a certain possibility of one or more blocks getting orphaned out at any time. So, the best strategy would be to wait untill many blocks are added to the chain. In other words, when a transaction receives multiple confirmations, it is safe to think that it is a part of the final consensus chain and will not get orphaned out. As any number of blocks get added after a certain block, that many number of confirmations are received by the transactions in that block. Though there are no rules as such that define how many confirmations one should get before accepting a transaction, six confirmations has been the best practice. Even with four confirmations, it is quite safe to assume a transaction has been confirmed, but six is the best practice because with more confirmations, the chances of a block getting orphaned out decreases exponentially.

Block Propagation

Bitcoin uses PoW mining to randomly select a node that can propose a valid block. Once the miner finds a valid block, they broadcast that block to the entire network. Block propagation in the network happens the same way as transactions. Every node that receives the new block further broadcasts it so that eventually the block reaches every node in the network. Please note that a node does not broadcast a block unless it is a part of the longest chain from its perspective. Once the nodes receive the new block that is proposed, they not only verify the header and check the hash value in acceptable range, but also validate each and every transaction that was included in that block. It is clear by now that for a node in the Bitcoin network, validating a block is a little more complex compared with validating transactions. Similar to transactions, there is a possibility that two valid blocks are proposed at the same time. In such a scenario, the node will keep both the blocks and start building on the one that comes from the longest chain.

We must understand that there is always a latency involved for a block to propagate through the entire network and reach every node. The relation between the block size and the time taken is linearly proportional, in the sense that for each kB added to the block size, the latency increases linearly. It is obvious that such network latency would impact the rate of growth of the blockchain. A measurement study that was conducted by Decker and Wattenhofer addresses this situation. Refer to Figure 3-19, which shows the relation between block size and the time it took to reach 25% (Line-1), 50% (Line-2), and 75% (Line-3) of monitored nodes.

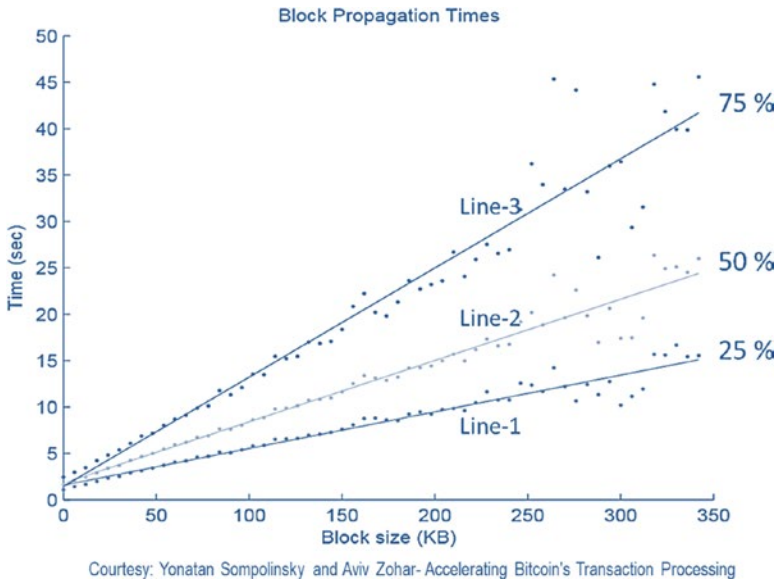


Figure 3-19. *Block propagation time with respect to block size*

The network bandwidth is the primary reason for such network latencies and it is never consistent in all areas of the globe. On top of this, we know that the broadcast packets of blocks go through many hops to finally reach all nodes. A typical Bitcoin block is of 1MB and the new variant of Bitcoin with a hard fork that has come up (Bitcoin Cash) is of 2MB block size; you can imagine the inherent limitations due to latency.

As per the network research, there are more than a million Bitcoin nodes that are connected to the Bitcoin network in a month and there are thousands of full nodes that are almost always connected to the network permanently.

Putting It all Together

At a high level, if we just put down the events in the order they take place, then here is how it may look:

- All new transactions are broadcast to all nodes.
- Each node that hears the new transactions collects them into a block.
- Each mining node works on finding a difficult PoW for its block to be able to propose it to the network.
- When a node gets lucky and finds a correct nonce to the PoW puzzle, it broadcasts the block to all nodes.
- Nodes accept the proposed block only if the nonce and all transactions in it are valid and not already spent.
- Bitcoin network nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash for the new block they would be mining.

Bitcoin Scripts

We learned about Bitcoin transactions in previous sections at a high level. In this section we will delve deep into the actual programming constructs that make the transactions happen. Every Bitcoin transactions' input and output are embedded with scripts. Bitcoin scripts are stack based,

which we will see shortly, and are evaluated from left to right. Remember that Bitcoin scripts are not Turing-complete, so you cannot really do anything and everything that is possible through other Turing-complete languages such as C, C++, or Java, etc. There are no concepts of loops in Bitcoin script, hence the execution time for scripts is not variable and is proportional to the number of instructions. This means that the scripts execute in a limited amount of time and there is no possibility for them to get stuck inside a loop. Also, most importantly, the scripts definitely terminate. Now that we know a little about the scripts, where do they run? Whenever transactions are made, whether programmatically, or through a wallet software or any other program, the scripts are injected inside the transactions and it is the work of the miners to run those scripts while mining. The purpose of Bitcoin scripts is to enable the network nodes to ensure the available funds are claimed and spent only by the entitled parties that really own them.

Bitcoin Transactions Revisited

A transaction in the Bitcoin network is a tranfer of value, which is a broadcast to the entire network and ends up in some block in the blockchain. Typically, it appears that Bitcoins get transferred from one account or wallet to another, but in reality, it is from one transaction to another. Well, before getting into further details, keep in mind that the Bitcoin addresses are basically the double-hashed output of the public key of the participants. The public key is first hashed using SHA256 and then by RIPEMD160 hashing algorithms, respectively, to generate 160-bit Bitcoin addresses. We have already covered these hashing techniques in the previous chapter. Let us zoom in a bit more into the transactions now. Take a look at the following transaction tree (Figure 3-20), the way it happens in Bitcoin.

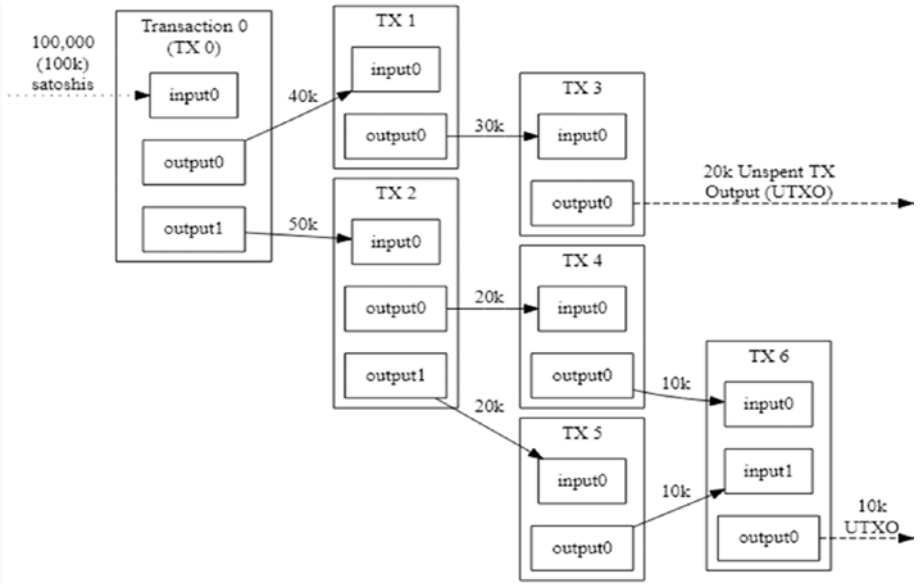


Figure 3-20. *A typical Bitcoin transaction structure*

Observe that the output of previous transactions become input to new transactions and this process continues forever. In the previous figure, if it was you who got the 100K from some previous output, it became the spendable input to a new transaction. Notice that in Tx 0, you spent 40K and 50K and paid up those amounts, and the remaining amount (10K) became the fee to the miner. By default, the remaining amount is paid to the miner, so you need to be careful not to ignore such situations, which are always the case. In this same situation, out of the remaining 10K amount, you could transfer say 9K to your own address and leave aside 1K for the mining fee. When an amount is not spent, in the sense that a transaction is not used as an input to a new transaction, it remains a UTXO, which can be spent later. Obviously, the ones previous to it are already spent. So, all the UTXOs combined for all the accounts (public keys) that you hold are your wallet balance.

Pause for a moment here, and think about how it must have been programmed. Remember that both the inputs and outputs of transactions are equipped with relevant scripts to make it possible. It is only through the scripts that it can be ensured that you are the authorized user to make a transaction and you have the necessary amount that you have received from a previous transaction. This means that both the inputs and outputs are equally important. Here is how the transaction's contents look:

```
Transaction Output = Amount of Bitcoins to transfer + Output
Script
Transaction Input = Reference to previous transaction output +
Input Script
```

Whether to look into the output script first or the input script first is actually an egg-chicken problem. But we will see the output script first because it is the one that is being consumed by the input script of the next transaction. Let us repeat and get this right, that while making a transaction, the output script of the current transaction is there just to enable the future transaction that can consume this as input, but for this current transaction, it is the previous transaction's output script that lets you spend it. This is why the output scripts have the public key of the recipient and the value (amount of Bitcoins) being transferred. When the output scripts are being used as inputs, their primary purpose is to check the signature against the public key. The output scripts are also called *ScriptPubKey*. Unless this output is spent, it remains a UTXO waiting to be spent.

The input script in the transaction input data structure has the mechanism of how to consume the previous transaction that you are trying to spend. So, it has to have the reference to that previous transaction. The hash of the previous transaction and the index number {hash, index} pinpoints the exact place where you had received the amount that you are now spending. The purpose of the "index" is to identify the intended output in the previous transaction. If you were the recipient of the previous

transaction, you have to provide your signature to claim that you are the rightful owner of the public key to which the transaction was made. This will let you spend that transaction. Also, you have to provide your public key, which will hash to the one used as destination address in previous transaction. Input scripts are also known as *ScriptSigs*. The ultimate objective of the script is to push the signatures and keys onto the stack.

A typical Bitcoin transaction has the following fields (Table 3-5).

Table 3-5. *Bitcoin Transaction Fields*

Field	Size	Description
Version no	4 bytes	Currently 1. It tells Bitcoin peers and miners which set of rules to use to validate this transaction.
In-counter	1 - 9 bytes	Positive integer (VI = VarInt). It indicates total number of inputs.
list of inputs	Variabe length	It lists all the inputs for a transaction.
Out-counter	1 - 9 bytes	Positive integer (VI = VarInt). It indicates total number of outputs.
list of outputs	Variable length	It lists all the outputs for a transaction.
lock_time	4 bytes	Not being used currently. It indicates if the transaction should be included in the blockchain block immediately after it is validated by the miner or there should be some lock time before it gets included in the block.

Let us now take a look at a different representation of the same transaction structure that we discussed in the previous section. This is to see a more detailed view of the transaction structure and the various components of it. Now refer to Figure 3-21.

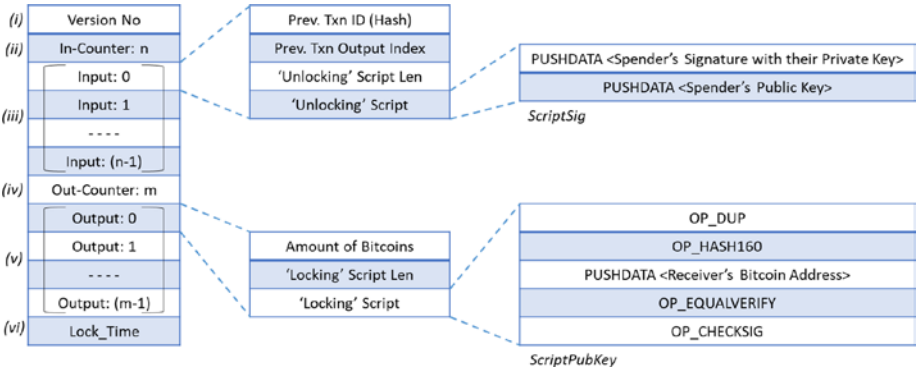


Figure 3-21. *Granular components of a Bitcoin transaction*

As you can see in the previous figure, the data items such as signatures or public keys are all embedded inside the scripts and are a part of the transaction. Just by looking at the granular components of Bitcoin transactions, many of your queries would be answered up front. The instructions in the script get pushed onto the stack and executed, which we wil explore in detail shortly.

When Bitcoin nodes receive the transaction broadcasts, they all validate those transactions individually, by combining the output script of the previous transaction with the input script of the current transaction following the steps mentioned as follows:

- Find the previous transaction whose output is being used as input to make this transaction. The "Prev. Txn ID (Hash)" field contains the hash of that previous transaction.
- In the previous transaction output, find the exact index where the amount was received. There could be multiple receivers in a transaction, so the index is used to identify the initiator of this current transaction whose address was used as recipient in the previous transaction.

- Consume the output script used in the previous transaction using the Unlocking Script called "*ScriptSig*." Notice in Figure 3-21 that there is a field before it that specifies the length of this Unlocking Script.
- Join this output script with the input script by just appending it to form the validation script and execute this validation script (remember this is a stack-based scripting language).
- The amount value is actually present in the Output Script, i.e., the "*ScriptPubKey*." It is the locking script that locks the transaction output with the spending conditions and ensures that only the rightful owner of the Bitcoin address to which this transaction has been made can later claim it. Observe that it also has the Locking Script Length field right before it. For the current transaction, this output script is only for information, and plays its role in the future when the owner tries to spend it.
- It is the validation script that decides if the current transaction input has the right to spend the previous UTXO by validating the signatures. If the validation script runs successfully, it is confirmed that the transaction is valid and the transaction went through.

Let us explore the previous explanation through a diagramatic representation to get a better understanding. Assume that Alice is paying Bob, say, five BTC. This means that Alice had received 5BTC in one of the previous transactions that was locked using *ScriptPubKey*. Alice can claim that she is the rightful owner of that transaction by unlocking it with *ScriptSig* and can spend it to Bob. Similarly, now if Bob tries to spend three

BTC to Charlie and two BTC to himself, then here is how it would look (Figure 3-22).

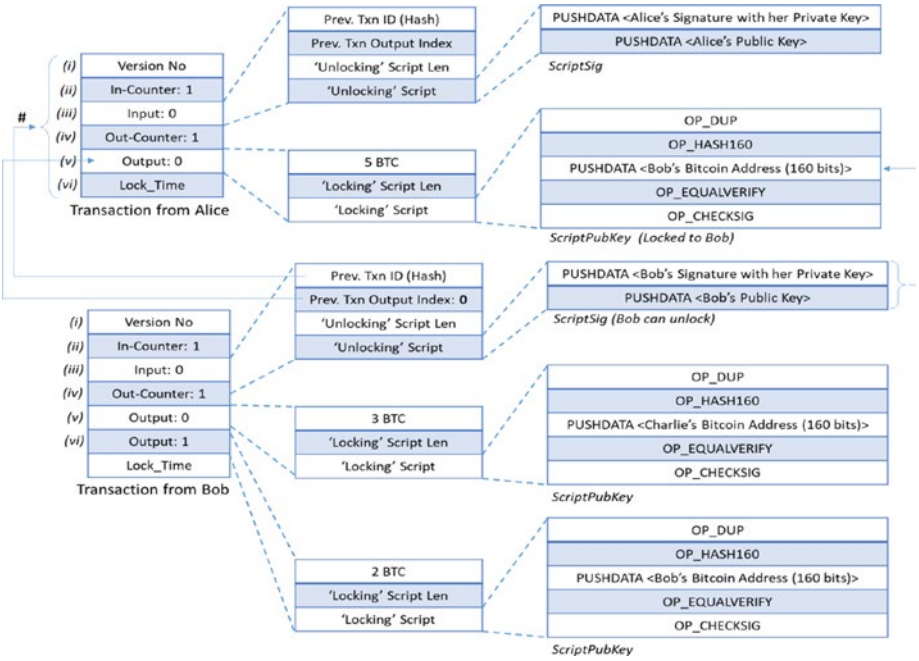


Figure 3-22. *A practical example of Bitcoin scripts*

When the Bitcoin network, more precisely the miners, receive the transaction from Alice, they check and confirm that it is a valid transaction and approve it by including it in their blocks (well, the one who proposes a block does it). When that happens, the output of this transaction becomes a part of the UTXO in the name of Bob, who could later spend it. And this is what happens in our example–that Bob also spends it to Charlie. Bob did so by consuming the previous transaction, unlocking it with his signature and public key, to prove that he is the owner of the Bitcoin address that Alice had used. Observe that there are two outputs in Bob's transaction. Since he had received five BTC from Alice and is paying three BTC to

Charlie, he must transfer the remainder to himself so that it becomes two BTC of UTXO bound to Bob himself and he could spend it in future. In Bob's transaction, the three BTC to Charlie is locked using the locking script for only Charlie to spend later.

Are you now thinking about how the scripts are combined and executed together? Remember that the unlocking script of current transaction is run along with the locking script of the previous transaction. As discussed already, running the scripts is a miner's job and they do not happen at the wallet software. In the previous example, when Bob makes the transactions, miners execute the *ScriptSig* unlocking script from Bob's transaction, and then immediately execute the *ScriptPubKey* locking script from Alice's transaction in order. If the sequential execution in a stack-based fashion for the combined validation script runs successfully by returning TRUE, then Bob's transaction is excepted by all the nodes validating it. We will take a look at Bitcoin scripts and how a Bitcoin-scripting virtual machine executes the stack during the execution of the combined script commands in more detail in the following section. In this section, however, take a look at the following example that represents the transaction from a developer's standpoint:

Example code with just one input and one output

```
{
  "hash": "a320ba8bbe163f26cafb2092306c153f87c1c2609b25db0
  c13664ae1afca78ce",
  "ver": 1,
  "vin_sz": 1,
  "vout_sz": 1,
  "lock_time": 0,
  "size": 51,
```

```
  "in":[
      {
         "prev_out":{
            "hash":"83cd5e9b704c0a4cb6066e3a1642b483adc8f73a76
            791c82a73dfa381281d32f",
            "n":0
         },
         "scriptSig":"63883d3d2dea35029d17d25b8a926675def004
         5c397d3df55b0ae145ef80db7849599b930220ab13bd2dda2
         ca0a67e2c5cd28030bb9b7b3dcacf176652dac82fe9d5873
         f3409661281d32f6d35b46906cd562bf8b48f4f938c077bcb
         29d46b0560fa5c61813d3d2d"
      }
   ],
  "out":[
      {
         "value":"0.08",
         "scriptPubKey":"OP_DUP OP_HASH160 b3a2c0d84ec82cff
         932b5c3231567a0d48ab4c78
OP_EQUALVERIFY OP_CHECKSIG"
      }
   ]
}
```


Note that Bitcoin transactions are not encrypted, so it is possible to browse and view every transaction ever collected into blocks.

Scripts

A script is actually a list of instructions recorded with each transaction that describes how the next person can gain access to those Bitcoins received and spend them. Bitcoin uses stack-based, non–Turing-complete scripting

language where the instructions get processed from left to right. Keep in mind that it is non–Turing-complete by design!

We looked at the input and output scripts in the previous section. We are now aware that the input script *ScriptSig* is the unlocking script and has two components, the public key and the signature. The public key is used because it hashes to the Bitcoin address that the transaction was spent to, in the previous transaction. The ECDSA digital signature's purpose is to prove the ownership of the public key, hence the Bitcoin address to be able to spend it further. Similarly, the output script *ScriptPubKey* in the previous transaction was to lock the transaction to the rightful owner of the Bitcoin address. These two scripts, *ScriptSig* of current transaction and *ScriptPubKey* of previous transaction, are combined and run. Take a look at its appearance after they are combined (Figure 3-23).

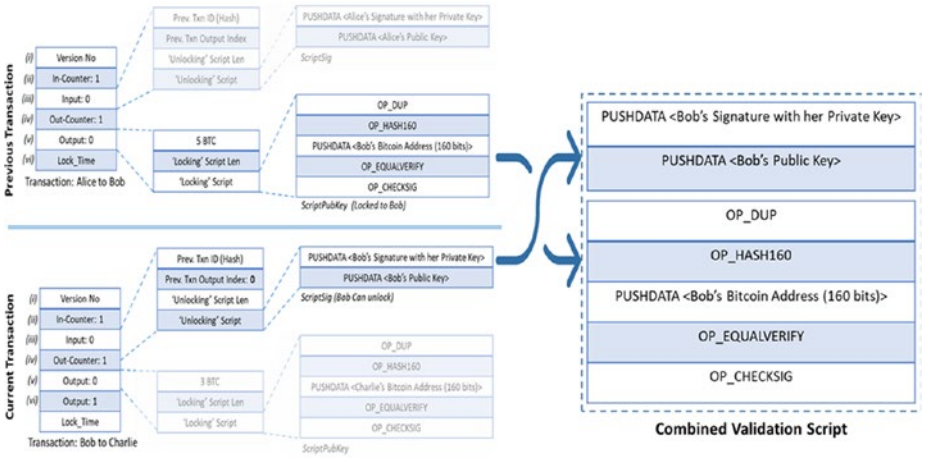


Figure 3-23. *Formation of combined validation script*

As we learned already, it is important to note that the Bitcoin script either runs successfully or it fails. When the transactions are valid, it runs successfully without any errors. Bitcoin scripting language is a very simplified version of programming languages and is quite small, with

just 256 instructions in total. Out of these 256, 15 are disabled and 75 are kept reserved maybe for later usage. These basic instructions comprise mathematical, logical (if/then), error reporting, and just return statements. Apart from these, there are some additional cryptographic instructions such as hashing, signature verification, etc. We will not get into all the available instruction sets, and focus only on the ones we will use in this chapter. Following are a few:

- **OP_DUP**: It just duplicates the top item in the stack.
- **OP_HASH160**: It hashes twice, first with SHA256 and then RIPEMD160.
- **OP_EQUALVERIFY**: It returns TRUE if the inputs are matched, and FALSE otherwise and marks the transaction invalid.
- **OP_CHECKSIG**: Checks if the input signature is a valid signature using the input Public Key itself for the hash of the current transaction.

To execute these instructions, we just have to push these instructions on to the stack and then execute. Apart from the memory that the stack takes, there is no extra memory required and this makes the Bitcoin scripts efficient. As you have seen, there are two kinds of instructions in the script, one is data instruction and the other is opcodes. The previous bullet list entries are all opcodes, and the combined validation script that we saw before has both of these kinds of instructions. Data instructions are just to push data onto the stack and not really to perform any function, and the sole purpose of opcodes is to execute some functions on the data in the stack and pop out as applicable. Let us discuss how Bob's transaction would get executed with such a stack-based implementation. Recollect the combined script where Bob is trying to spend a previously received transaction in the current transaction to Charlie (Figure 3-24).

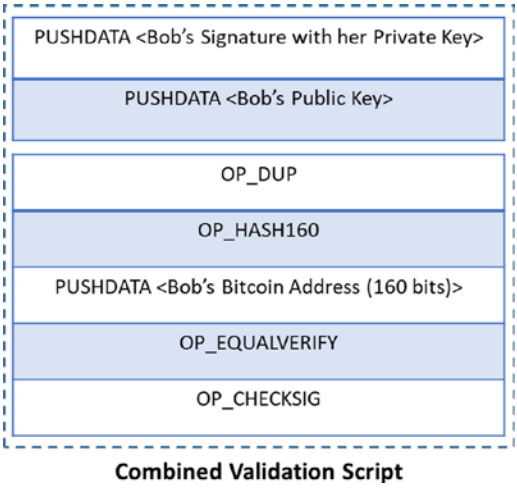


Figure 3-24. *Combined script of ScriptPubKey and CheckSig*

The corresponding stack-based implemention would be as follows (Figure 3-25).

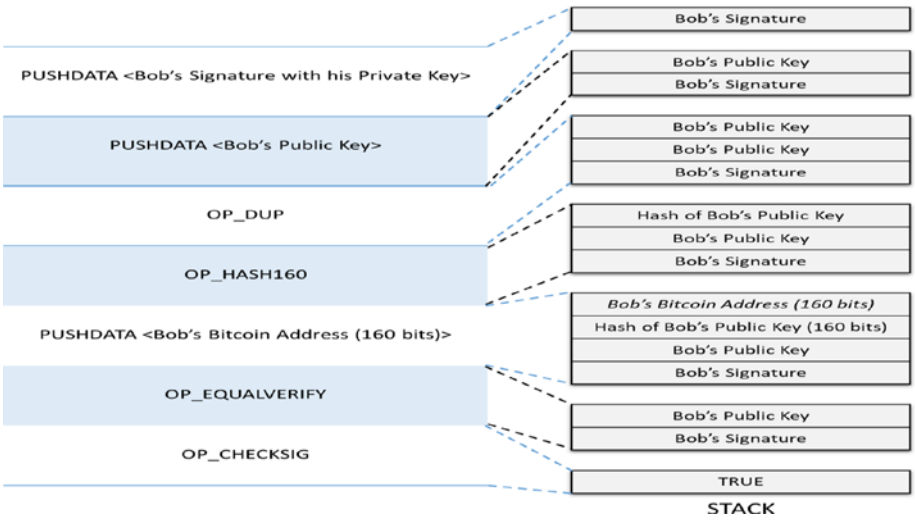


Figure 3-25. *Example of stack-based implementation of Bitcoin script*

Though the previous stack-based implementation is self explanatory, we will quickly run through what happened here.

- First was Bob's signature–a data instruction and so was pushed onto the stack
- Then was his public key–again a data instruction and was pushed on to the stack
- Then it was OP_DUP, an opcode. It duplicates the first item in the stack, so the public key of Bob was duplicated and became the third item on the stack.
- Next was OP_HASH160, an Opcode, which hashed Bob's public key twice, once with SHA256 and then with RIPEMD160, and the final 160 bits output replaced Bob's public key and became the top of the stack.
- Then it was Bob's Bitcoin address (160 bits)–a data instruction, which was pushed to the stack.
- Next was an opcode, OP_EQUALVERIFY, which checks the top two items in the stack and if they match, it pops them both else an error is thrown and the script would terminate.
- Then was again an opcode OP_CHECKSIG, which checks the public key against the signature to validate the authenticity of the owner. This opcode is also capable of consuming both inputs and popping them off the stack.

You must be wondering what if someone tries to inject some fraudulent scripts or tries to misuse them. Please note that Bitcoin scripts are standardized and the miners are aware of them. Anything that does not follow the standard gets dropped by the miners, as they wouldn't waste their time executing such transactions.

Full Nodes vs. SPVs

We already got a heads-up on the full nodes and SPVs in this chapter. It is obvious that the notion of full node and lightweight node is implemented to ease out the usage of Bitcoins and make them more adaptable. In this section, we will zoom in to the technicalities for these variants and understand their purpose.

Full Nodes

The full nodes are the most integral components in a Bitcoin network. They are the ones that maintain and run Bitcoin from various places across the globe. As discussed already, download the entire blockchain with all transactions, starting all the way from the genesis block to the latest discovered block. The latest block defines the height of the blockchain.

The full nodes are extremely secure because they have the entire chain. For an adversary to be successful in cheating a node, an alternative blockchain needs to be presented, which is practically impossible. The true chain is the most cumulative PoW chain, and it gets computationally infeasible to propose a new fraudulent block. If all transactions are not valid in a block, PoW mining performed by the adversary will be in vain, because other miners will not mine on top of it. Such a block gets orphaned out soon enough. Full nodes build and maintain their own copy of blockchain locally. They do not rely on the network for transaction validation because they are self-sufficient. They are just interested in knowing the new blocks that get proposed by other nodes so that they can update their local copy after validating blocks. So, we learned that each full node must process all transactions; they must store the entire database, every transaction that is currently being brioadcast, every transaction that is ever spent, and the list of UTXOs; participate in maintaining the entire Bitcoin network; and they also have to serve the SPV clients.

Note that there are so many varities of Bitcoin software that the full nodes use that are quite different in software architecture and programmed in different language constructs. However, the most widely used one is the "Bitcoin Core" software; more than three fourths of the network uses it.

SPVs

Bitcoin design has this nice concept of Simple Payment Verification(SPV) nodes that can be used to verify transactions without running full nodes. The way SPVs work is that they download only the header of all the blocks during the initial syncing to the Bitcoin network. In Bitcoin, the block headers are of 80 bytes each, and downloading all the headers is not much and ranges to a few MBs in total.

The purpose of SPVs is to provide a mechanism to verify that a particular transaction was in a block in a blockchain without requiring the entire blockchain data. Every block header has the Merkle root, which is the block hash. We know that every transaction has a hash and that transaction hash can be linked to the block hash using the Merkle tree proof which we discussed in the previous chapter. All the transactions in a block form the Merkle leafs and the block hash forms the Merkle root. The beauty of the Merkle tree is that only a small part of the block is needed to prove that a transaction was actually a part of the block. So, to confirm a transaction an SPV does two things. First, it checks the Merkle tree proof for a transaction to accertain it is a part of the block and second, if that block is a part of the main chain or not; and there should be at least six more blocks created after it to confirm it is a part of the longest chain. Figure 3-26 depicts this process.

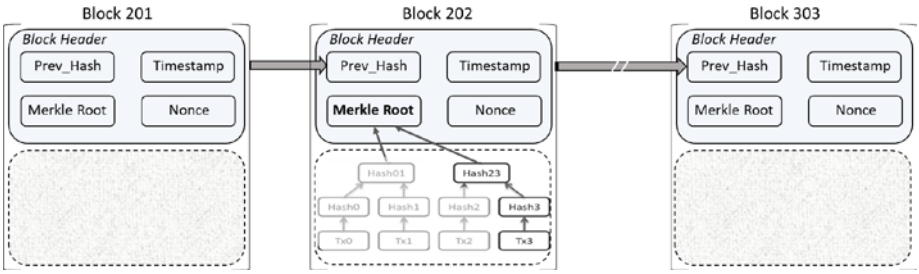


Figure 3-26. *Merkle root in block header of SPV*

Let us dig deeper into the technicality of how SPVs really work in verifying transactions. At a high level, it takes the following steps:

- To the peers an SPV is connected to, it establishes Bloom filters with many of them and ideally not just to one peer, because there could be a chance for that peer to perform denial of service or cheat. The purpose of Bloom filters is to match only the transactions an SPV is interested in, and not the rest in a block without revealing which addresses or keys the SPV is interested in.
- Peers send back the relevant transactions in a *merkleblock* message that contains the Merkle root and the Merkle path to the transaction of interest as shown in the figure above. The *merkleblock* message sze is in a few kB and quite efficient.
- It is then easy for the SPVs to verify if a transaction truly belongs to a block in the blockchain.
- Once the transaction is verified, the next step is to check if that Block is actually a part of the true longest blockchain.

The following (Figure 3-27) represents this SPV communication steps with its peers.

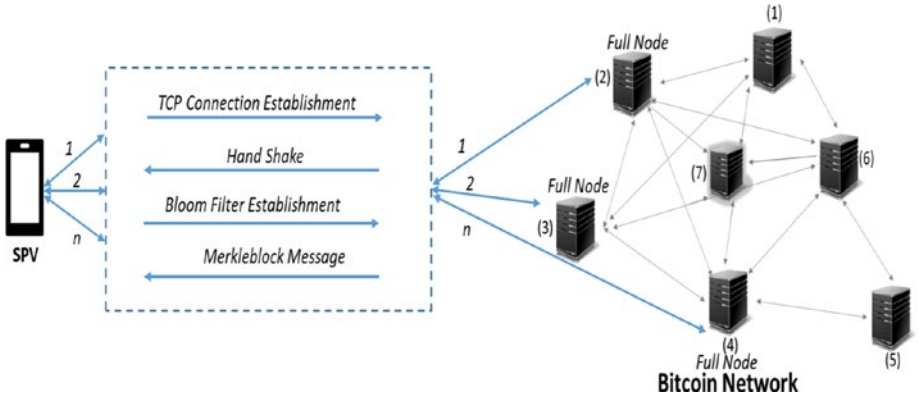


Figure 3-27. *SPV communication mechanism with the Bitcoin network*

Bitcoin Wallets

Bitcoin wallets are very similar to the wallet you use in your daily life, in the sense you have access to it and you can spend when you want. Bitcoin wallets, however, are a digital phenomenon. Recollect the example we used in the previous section, where Alice paid some amount to Bob. How would she do it if Bob did not have an account? In the Bitcoin setting, the accounts or wallets are represented by the Bitcoin address. Bob must first generate a key pair (private/public keys). Bitcoin uses the ECDSA algorithm with *secp256k1* curve (don't worry, it is just the curve type–a standard recommendation). First a random bit string is generated to serve as private key, which is then deterministically transformed to public key. As we learned before in Chapter 2, the private/public keys are mathematically related and the public key can be generated from the private key any time (deterministic). So, it is not really a requirement to save the public keys. as such. True randomness is not possible through

software implementations, so many servers or applications use hardware security modules (HSMs) to generate true random bits and also to protect the private keys. Unlike public keys, private keys definitely require saving them with maximum security. If you lose them, you cannot generate a signature that would justify the ownership of the public key (or Bitcoin address) that received some amount in any transaction. The public keys are hashed twice to generate the Bitcoin adress, first with SHA256 and then with RIPEMD160. This is also deterministic, so given a public key, it is just a matter of a couple of hashes to generate the Bitcoin address.

Note carefully that the Bitcoin address does not really reveal the public key. This is because the addresses are double-hashed public keys and it's quite infeasible to find the public key given the Bitcoin address. However, for someone with a public key, it is easy to claim the ownership of a Bitcoin address. The hashing technique in Bitcoin shortens and obfuscates the public key. While it makes the manual transcription easier, it also provides security against unanticipated problems that might allow reconstruction of private keys from public keys. This is possibly the safest implementation! Public keys are revealed only when the transaction output is being claimed by the owner, not when it was transacted to them, as you can see in Figure 3-28.

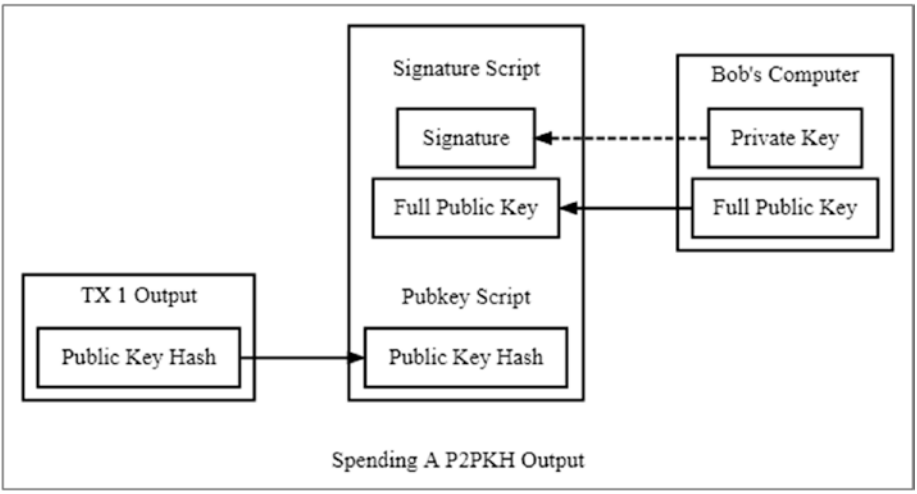


Figure 3-28. *Revealing public key to claim a transaction*

Bitcoin wallets are nothing but the SPVs and are served by the full nodes. We already looked at the functioning of SPVs, so in this section we will take a look at some wallet-specific activities. We all understand that to make a transaction, or to receive a transaction, you need not be running a full node. All you want is a wallet to be able to save your private/public key pair, to be able to make and receive transactions (actually view and verify the ones made to you). We already learned the verification part while going through the SPVs section. Let us take a look at how to initiate a transaction using a wallet.

It is advisible that you run your own full node and connect your wallet to it, as it would be the most secured way of working on Bitcoin. However, it is not a mandate and you can still work without maintaining your own node. Keep in mind that when you query a node, you have to mention your public address to get the list of UTXOs, and the full node is becoming aware of your public address, which is a privacy leak! All a wallet has to do is get the list of UTXOs so it can spend a transaction by signing it with its private key and publish that transaction into the Bitcoin network. This can be done by creating your own wallet software or by using a third-party

wallet service. However, be careful with the wallet service providers because you are allowing them to take control of your private key. Whether they deliberately take your Bitcoins or they themselves are hacked, which has been the case with many wallet services, you lose your Bitcoins. At the end of the day, all wallet-service providers are centralized, though the Bitcoin network is decentralized. A typical pictorial representation of initiating a Bitcoin transaction through the wallet software can be represented as shown in the following (Figure 3-29).

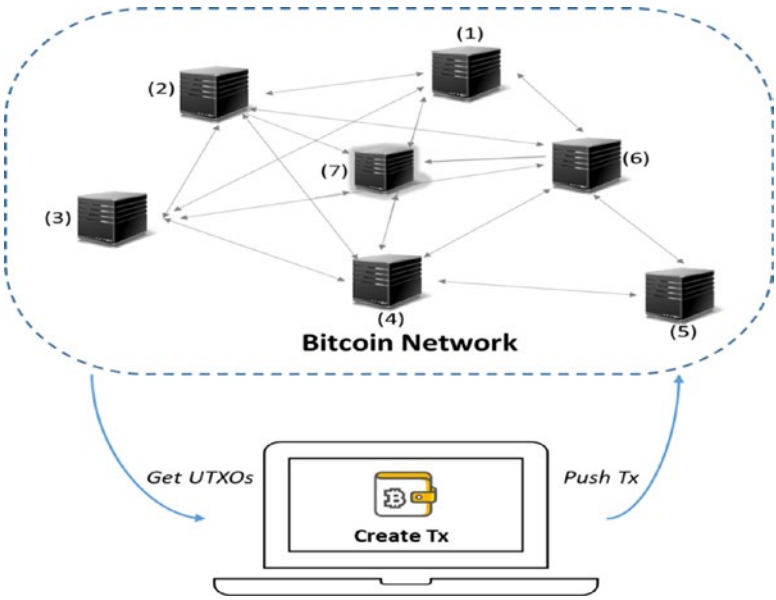


Figure 3-29. *A wallet application interacting with the Bitcoin network*

An example of an SPV client that can serve as a Bitcoin wallet is "BitcoinJ." BitcoinJ is actually a library to work with the Bitcoin protocol, maintain a wallet, and initiate/validate transactions. It does not require a full node such as a Bitcoin Core node locally and can function as a thin client node. Though it is implemented in Java, it can be used from any JVM-compatible language such as JavaScript and Python.

Summary

In this chapter, we learned how blockchain concepts we discussed in the previous chapter were put together to build Bitcoin as a cryptocurrency use case of blockchain technology. We covered the evolution of Bitcoin, the history of it, what it is, the design benefits, and why it is so important. We got to know about granular details on the Bitcoin network, transactions, blocks, the blockchain, consensus, and how all these are stitched together. Then we learned about the requirement of a wallet solution to interact with the Bitcoin blockchain system.

In the 1990s, mass adoption of the Internet changed the way people did business. It removed friction from creation and distribution of information. This paved the way for new markets, more opportunities, and possibilities. Similarly, blockchain is here today to take the Internet to a whole new level. Bitcoin is just one cryptocurrency application of blockchain, and the possibilities are limitless. In the next chapter, we will learn about how Ethereum works and how it has become a defacto standard for various decentralized applications on one public blockchain network.

References

Bitcoin: A Peer-to-Peer Electronic Cash System

Nakamoto, Satoshi, "Bitcoin: A Peer-to-Peer Electronic Cash System," https://bitcoin.org/bitcoin.pdf.

All about Bitcoin Network and Transactions

Bitcoin wiki, https://en.bitcoin.it/.

Blockchain Technology

Crosby, Michael, Nachiappan; Pattanayak, Pradhan, Verma, Sanjeev, Kalyanaraman, Vignesh, "BlockChain Technology: Beyond Bitcoin," Sutardja Center for Entrepreneurship & Technology, University of California, Berkeley, http://scet.berkeley.edu/wp-content/uploads/BlockchainPaper.pdf, October 16, 2015.

Accelerating Bitcoin's Transaction Processing

Sompolinsky, Yonatan, Zohar, Aviv, "Secure High-Rate Transaction Processing inBitcoin," Hebrew University of Jerusalem, Israel, School of Engineering and Computer Science, https://eprint.iacr.org/2013/881.pdf.